**Szegedi Tudományegyetem**
**Természettudományi és Informatikai Kar**
**Informatikai Intézet**

SZAKDOLGOZAT

# Lightweight kriptográfiai algoritmusok teljesítmény-elemzése erőforrás-korlátozott rendszerekre

Performance Analysis of Lightweight Cryptographic Algorithms for Resource-Constrained Systems

*Készítette:*
**Kárász Barna Kristóf**
Mérnökinformatikus BSc hallgató

*Témavezető:*
**Dr. Békési József**

Szeged
2026

**Kivonat**

Az IoT-eszközök jelentős része kevés memóriával, alacsony órajellel és szűkös energiával működő mikrovezérlő, amelyen a megszokott kriptográfiai megoldások futtatása költséges. A dolgozat négy szimmetrikus kriptográfiai algoritmust hasonlít össze egy ARM Cortex-M4 magos STM32L432KC mikrovezérlőn: az AES-128-at mint viszonyítási alapot, a SPECK 128/128 és a LEA-128 könnyűsúlyú blokktitkosítót, valamint az ASCON-128a hitelesített titkosítási sémát, amelyre a NIST 2025-ben kiadott könnyűsúlyú szabványa épül. A publikált referencia-implementációkat a célplatformra adaptáltuk, és a processzormag ciklusszámlálójával mértük a kulcsgenerálás, a titkosítás és a visszafejtés költségét 16 és 1024 bájt közötti üzenetméreteken, négy fordítói optimalizációs szinten. A mért ciklusszámokból a fajlagos költséget és az átviteli sebességet is meghatároztuk. `-O3` optimalizáció mellett az ARX-alapú SPECK és LEA a vizsgált AES-implementációnál titkosításkor 3,7–4,5-ször, visszafejtéskor 8,7–10-szer kisebb fajlagos költséggel dolgozott, az ASCON-128a pedig a hitelesítéssel együtt is gyorsabb volt az AES-nél, ha az üzenet nagyjából 100 bájtnál hosszabb. Az algoritmusok biztonsági jellemzőit a szakirodalom alapján foglaljuk össze, végül a mérések alapján alkalmazási ajánlásokat fogalmazunk meg.

**Kulcsszavak:** könnyűsúlyú kriptográfia, IoT-biztonság, blokktitkosító, AES, SPECK, LEA, ASCON, Cortex-M4, teljesítménymérés

# Tartalomjegyzék

**1. Bevezetés** **1**

**2. Háttérismeretek** **3**

2.1. Szimmetrikus kriptográfia alapjai . . . . . 3

2.1.1. Blokktitkosítók . . . . . 3

2.1.2. Üzemmódok . . . . . 5

2.1.3. A DES és az AES rövid áttekintése . . . . . 6

2.2. Hashfüggvények és üzenethitelesítés . . . . . 6

2.2.1. Kriptográfiai hashfüggvények . . . . . 6

2.2.2. MAC – üzenethitelesítő kód . . . . . 7

2.2.3. AEAD – hitelesített titkosítás kapcsolódó adatokkal . . . . . 7

2.3. Erőforrás-korlátozott rendszerek jellemzői . . . . . 7

2.3.1. Mikrovezérlők és erőforrásaik . . . . . 7

2.4. A lightweight kriptográfia motivációja . . . . . 8

2.4.1. A hagyományos szabványok korlátai . . . . . 8

2.4.2. A NIST Lightweight Cryptography szabványosítás . . . . . 9

**3. Vizsgált algoritmusok bemutatása** **10**

3.1. AES-128 . . . . . 10

3.1.1. Felépítés és működés . . . . . 10

3.1.2. Kulcsgenerálás . . . . . 11

3.1.3. Az AES-128 mint viszonyítási alap . . . . . 11

3.2. SPECK 128/128 . . . . . 11

3.2.1. Körfüggvény . . . . . 11

3.2.2. Kulcsgenerálás . . . . . 12

3.2.3. Szoftveres megvalósítás . . . . . 12

3.2.4. Bizalmi kérdések . . . . . 12

3.3. LEA-128 . . . . . 13

3.3.1. Általánosított Feistel-hálózat . . . . . 13

3.3.2. A 32 bites szóméret . . . 13
3.3.3. Kulcsgenerálás . . . 13
3.4. ASCON-128 . . . 14
3.4.1. Szivacs- és duplexkonstrukció . . . 14
3.4.2. Az ASCON-permutáció . . . 14
3.4.3. Az AEAD működése . . . 15
3.4.4. A vizsgált változat . . . 15
3.5. Az algoritmusok összevetése . . . 16

**4. Implementáció és mérési módszer** **17**
4.1. A mérési platform . . . 17
4.2. A forráskód szerkezete . . . 18
4.3. Az algoritmusok megvalósítása . . . 18
4.3.1. AES-128 . . . 19
4.3.2. SPECK 128/128 . . . 19
4.3.3. LEA-128 . . . 20
4.3.4. ASCON-128 . . . 20
4.4. A mérési keretrendszer . . . 20
4.4.1. Önellenőrzés és bemenetek . . . 20
4.4.2. Ciklusmérés a DWT-egységgel . . . 20
4.4.3. Ismétlések és statisztika . . . 21
4.4.4. Az eredmények kiírása . . . 22
4.5. Az üzemmódok megválasztása . . . 22
4.6. Fordítási beállítások . . . 22

**5. Teljesítménymérések és eredmények** **24**
5.1. A kulcsgenerálás költsége . . . 24
5.2. Titkosítás és visszafejtés . . . 25
5.3. Skálázódás az üzenetmérettel . . . 26
5.4. Feldolgozási sebesség . . . 28
5.5. Kódméret és memóriaigény . . . 29
5.6. A fordítói optimalizáció hatása . . . 30
5.6.1. Optimalizálás nélkül és optimalizálva . . . 30
5.6.2. `-O2`, `-O3` és `-Ofast` . . . 31
5.6.3. `-Os/-Oz` . . . 31
5.6.4. A mérések ismételhetősége . . . 32
5.7. Az eredmények összegzése . . . 32

**6. Biztonsági elemzés és összehasonlító értékelés** **33**
6.1. Alapfogalmak . . . 33
6.1.1. Támadási modellek . . . 33
6.1.2. Biztonsági szint és biztonsági tartalék . . . 33
6.2. A főbb támadástípusok . . . 34
6.2.1. Differenciális kriptoanalízis . . . 34
6.2.2. Lineáris kriptoanalízis . . . 34
6.2.3. Algebrai és rotációs támadások . . . 34
6.2.4. Oldalcsatorna-támadások . . . 35
6.2.5. Születésnapi korlát . . . 35
6.3. Az algoritmusok biztonsági jellemzői . . . 35
6.3.1. AES-128 . . . 35
6.3.2. SPECK 128/128 . . . 36
6.3.3. LEA-128 . . . 36
6.3.4. ASCON-128 . . . 36
6.4. Összehasonlító értékelés . . . 36

**7. Összefoglalás és továbbfejlesztési lehetőségek** **38**
7.1. A célkitűzések teljesülése . . . 38
7.2. Fő megállapítások . . . 38
7.3. Alkalmazási ajánlások . . . 39
7.4. Korlátok . . . 40
7.5. Továbbfejlesztési lehetőségek . . . 40

**Irodalomjegyzék** **42**

**Köszönetnyilvánítás** **45**

**Nyilatkozat** **46**

**Mellékletek** **47**

# 1. fejezet

# Bevezetés

Az elmúlt évtizedben a hálózatba kapcsolt eszközök az ipar, a közlekedés és a háztartások mindennapi részévé váltak. Az *Ipar 4.0* keretében a gyártósorok, a logisztikai láncok és az épületgépészeti rendszerek egyre inkább egymással kommunikáló eszközökre épülnek [25]. A dolgok internete (Internet of Things, IoT) ma már nem csupán kutatási téma: okos termosztátok, viselhető egészségügyi eszközök, mezőgazdasági szenzorok és közlekedésirányító rendszerek alkotják.

Az Ericsson jelentése szerint 2025 végén világszerte 22,3 milliárd IoT-kapcsolat volt, és ez a szám 2031-re megközelítheti a 47,1 milliárdot [24]. Ezeknek az eszközöknek jelentős része nem számítógép vagy okostelefon, hanem mikrovezérlő, szenzor vagy RFID-címke, amely kevés memóriával, alacsony órajellel és szűkös, gyakran elemről biztosított energiaellátással működik.

Ezek az eszközök sokszor érzékeny adatokat kezelnek. Egy gyártósoron a meghamisított szenzorérték vagy vezérlőparancs anyagi és fizikai kárt okozhat, egy közlekedési rendszerben a módosított adat balesethez vezethet, egy egészségügyi eszköznél pedig a kiszivárgó betegadat súlyos adatvédelmi következményekkel jár. A kommunikáció titkosítása és hitelesítése ezért ezeken az eszközökön is alapkövetelmény [23].

A széles körben használt kriptográfiai eljárásokat nem elsősorban ilyen eszközökre tervezték. Az aszimmetrikus algoritmusok (pl. RSA) egy kis mikrovezérlőn lassúak és sok energiát fogyasztanak, de még a szimmetrikus AES is érezhető részt foglalhat el a legszűkebb eszközök memóriájából és számítási idejéből. Erre a problémára kínál megoldást a *könnyűsúlyú* (lightweight) *kriptográfia*, amely kevés erőforrással is az adott alkalmazáshoz elegendő biztonságot igyekszik nyújtani [6].

A terület jelentőségét jól mutatja, hogy a NIST (National Institute of Standards and Technology) 2018-ban külön szabványosítási eljárást indított a könnyűsúlyú kriptográfiára, és 2023-ban a grazi kutatók által tervezett ASCON-t választotta ki szabványnak [10].

A dolgozatban négy, eltérő felépítésű kriptográfiai algoritmust hasonlítunk össze. Célunk annak megmérése, hogy az egyes algoritmusok mekkora futásidejű költséggel járnak egy erőforrás-korlátozott eszközön. A méréseket egy ARM Cortex-M4 magos STM32L432KC mikrovezérlőn végeztük: a kulcsgenerálás, a titkosítás és a visszafejtés ciklusszámát mértük, ebből pedig a fajlagos költséget és az átviteli sebességet határoztuk meg, több fordítói optimalizációs szinten. A vizsgált algoritmusok:

1. **AES-128** – az iparban legelterjedtebb szimmetrikus blokktitkosító, amely az összehasonlításban viszonyítási alapként szolgál;
2. **SPECK 128/128** – az NSA által tervezett, szoftveres futásra optimalizált, ARX-alapú könnyűsúlyú blokktitkosító;
3. **LEA-128** – a dél-koreai NSRI által kidolgozott könnyűsúlyú blokktitkosító, amely az ISO/IEC 29192-2:2019 szabvány része;
4. **ASCON-128a** – a NIST által 2023-ban kiválasztott ASCON család szivacskonstrukcióra épülő hitelesített titkosítási sémája.

Az algoritmusokat úgy választottuk ki, hogy mindegyik más tervezési elvet képviseljen (SPN, ARX-alapú Feistel-hálózat, ARX-alapú általánosított Feistel-hálózat, illetve szivacskonstrukció), ugyanakkor összevethetők maradjanak: mind a négy 128 bites kulcsot használ, a blokktitkosítók 128 bites blokkot, és mindegyik jól megvalósítható szoftverben. Így a mért különbségeket nemcsak közölni, hanem az algoritmusok felépítésével magyarázni is tudjuk.

A dolgozat felépítése a következő. A 2. fejezet a szimmetrikus kriptográfia alapjait, a hashfüggvényeket és az üzenethitelesítést, az erőforrás-korlátozott rendszerek jellemzőit, valamint a könnyűsúlyú kriptográfia megjelenésének okait mutatja be. A 3. fejezet a négy kiválasztott algoritmust ismerteti és veti össze. A 4. fejezet a szoftveres megvalósítást és a mérési módszert, a 5. fejezet a mérési eredményeket tárgyalja. A 6. fejezet a szakirodalom alapján összeveti az algoritmusok biztonsági jellemzőit, végül a 7. fejezet összegzi az eredményeket, alkalmazási ajánlásokat fogalmaz meg, és továbbfejlesztési lehetőségeket vázol fel.

# 2. fejezet

# Háttérismeretek

Ez a fejezet azokat az alapokat foglalja össze, amelyekre a dolgozat további része épül: a szimmetrikus kriptográfia fő fogalmait, a hashfüggvényeket és az üzenethitelesítést, az erőforrás-korlátozott eszközök jellemzőit, valamint azokat a gyakorlati okokat, amelyek miatt a könnyűsúlyú kriptográfia önálló kutatási területté vált.

## 2.1. Szimmetrikus kriptográfia alapjai

A *szimmetrikus kriptográfiában* a titkosítás és a visszafejtés *ugyanazzal a titkos kulccsal* történik. Formálisan: adott egy $E$ titkosító és egy $D$ visszafejtő függvény, amelyekre tetszőleges $P$ nyílt szöveg és $K$ kulcs esetén $D(K, E(K, P)) = P$ [1]. Az aszimmetrikus (nyilvános kulcsú) eljárásoknál lényegesen kevesebb számítást igényelnek, ezért erőforrás-korlátozott eszközökön elsősorban ezeket használják az adatok titkosítására.

### 2.1.1. Blokktitkosítók

A szimmetrikus titkosítás leggyakoribb formája a *blokktitkosító* (block cipher). Ez a bemenetet rögzített méretű, jellemzően 64 vagy 128 bites egységekre, úgynevezett *blokkokra* bontja, és mindegyiket a kulcs függvényében egy azonos méretű titkosított blokká alakítja [2]. A blokktitkosító több *körből* (round) áll. Egy-egy kör önmagában egyszerű műveleteket végez, de kellő számú ismétlés után a kimenet már nem mutat kihasználható összefüggést a bemenettel és a kulccsal. A tervezés két alapelvét Claude Shannon fogalmazta meg 1949-ben [3]:

- **Konfúzió** (confusion): a titkosított szöveg és a kulcs közötti kapcsolatnak a lehető legbonyolultabbnak kell lennie. Ezt nemlineáris műveletek biztosítják, például helyettesítő táblák (S-boxok) vagy moduláris összeadás.

– **Diffúzió** (diffusion): a nyílt szöveg egyetlen bitjének megváltozása a titkosított szöveg minél több bitjére hasson ki. Ezt permutációk, bitforgatások vagy lineáris transzformációk (pl. véges test feletti mátrixszorzás) érik el.

A következőkben a blokktitkosítók három gyakori felépítését mutatjuk be. A dolgozatban vizsgált blokktitkosítók mindegyike ezek valamelyikére épül.

### SPN – helyettesítő-permutációs hálózat

A *Substitution-Permutation Network* (SPN) felépítésű blokktitkosítók minden körben három réteget alkalmaznak. A *helyettesítő réteg* (substitution layer) a blokkot kisebb, általában 4 vagy 8 bites részekre bontja, és mindegyiket egy rögzített nemlineáris leképezésen, az *S-boxon* vezeti át. Az S-box felel elsősorban a konfúzióért.

A második, *diffúziós réteg* szétteríti az S-boxok kimenetét a blokkon, így a következő körben minden S-box több előző S-box eredményét kapja meg. Egyszerűbb SPN-eknél ez a bitek átrendezése (permutációja), az AES-nél viszont egy lineáris transzformáció (ShiftRows és MixColumns). Néhány kör után így a nyílt szöveg egyetlen bitjének változása a teljes kimeneti blokkot érinti. Végül a *körkulcs hozzáadása* (round key addition) során egy XOR művelet keveri az állapotba a fő kulcsból származtatott körkulcsot. Az SPN legismertebb képviselője az AES [5].

### Feistel-hálózat

A Horst Feistelről elnevezett szerkezet a blokkot két egyenlő félre, $L$-re (bal) és $R$-re (jobb) bontja. Minden körben az $R$ félre egy $F$ körfüggvényt alkalmaz az aktuális körkulccsal, az eredményt XOR-ral hozzáadja az $L$ félhez, majd a két felet felcseréli:

$$\begin{aligned} L_{i+1} &= R_i, \\ R_{i+1} &= L_i \oplus F(R_i, K_i). \end{aligned} \tag{2.1}$$

A Feistel-hálózat nagy előnye, hogy a visszafejtés ugyanazzal a szerkezettel történik, mint a titkosítás, csak a körkulcsokat kell fordított sorrendben alkalmazni. Ezért az $F$ függvénynek nem kell invertálhatónak lennie [2]. A szerkezet legismertebb példája a DES (Data Encryption Standard), amelynek 16 körös Feistel-hálózata 1977-től évtizedekig meghatározó szabvány volt. 56 bites kulcsa azonban ma már nem véd a kulcstér végigpróbálása (brute force) ellen.

### ARX-szerkezet

Az *Add-Rotate-XOR* (ARX) megközelítés három egyszerű műveletre épül [12]:

– **Add** – moduláris összeadás (mod$2^n$): az átvitelek terjedése miatt ez a nemlineáris elem, amely S-box nélkül biztosítja a konfúziót;

– **Rotate** – bitforgatás (ciklikus eltolás): a szón belüli diffúziót szolgálja;

– **XOR** – kizáró vagy: a körkulcs bekeverésére és az ágak összekeverésére szolgál.

Az ARX fő előnye az egyszerűség. Ha a szóméret megegyezik a processzor regiszterszélességével, mindhárom művelet egyetlen gépi utasítás, és nincs szükség memóriában tárolt helyettesítő táblára. Mivel a műveletek végrehajtási ideje nem függ a feldolgozott adattól, az ARX-algoritmusok könnyen megvalósíthatók *konstans idejű* (constant-time) kóddal, ami fontos védelem az időzítéses oldalcsatorna-támadások (timing attacks) ellen. A dolgozatban vizsgált SPECK és LEA egyaránt ARX-alapú, de eltérő szerkezetben használja ezeket a műveleteket.

### 2.1.2. Üzemmódok

A blokktitkosító egyszerre csak egy rögzített méretű blokkot dolgoz fel, a valódi üzenetek hossza viszont tetszőleges. Azt, hogy az egymást követő blokkokat hogyan titkosítjuk és hogyan kapcsoljuk össze, az *üzemmód* (mode of operation) határozza meg. A legfontosabbak [1]:

– **ECB** (Electronic Codebook): minden blokkot egymástól függetlenül, ugyanazzal a kulccsal titkosítunk. Egyszerű és párhuzamosítható, de az azonos nyíltszöveg-blokkokból mindig azonos titkosított blokkok lesznek, így az adat mintázatai (például egy kép körvonalai) a titkosított szövegben is látszanak. Ezért önmagában nem ajánlott.

– **CBC** (Cipher Block Chaining): titkosítás előtt minden nyíltszöveg-blokkot XOR-ral összekapcsolunk az *előző* titkosított blokkal; az első blokkhoz egy véletlenszerű inicializációs vektort (IV) használunk. A láncolás miatt azonos bemeneti blokkokból különböző kimenet keletkezik. Hátránya, hogy a titkosítás nem párhuzamosítható.

– **CTR** (Counter Mode): a blokktitkosító egy minden blokknál más számlálóértéket titkosít, és az így kapott *kulcsfolyamot* (keystream) XOR-ral adjuk a nyílt szöveghez. A számlálóértékek előre ismertek, ezért a titkosítás és a visszafejtés is párhuzamosítható, a visszafejtéshez pedig elég a titkosító függvény. Lényeges feltétel, hogy ugyanazzal a kulccsal egy számlálóérték soha ne ismétlődjön.

### 2.1.3. A DES és az AES rövid áttekintése

A *Data Encryption Standard* (DES) az 1977-es szabványosítás után évtizedekig a legszélesebb körben használt szimmetrikus algoritmus volt. Feistel-hálózata 16 körből áll, a nemlineáris lépést 8 darab, 6 bites bemenetet 4 bites kimenetre képező S-box végzi. A szabvány bukását az 56 bites kulcs okozta: 1998-ban az Electronic Frontier Foundation (EFF) erre a célra épített gépe 56 óra alatt találta meg a kulcsot, 1999-ben pedig a distributed.net önkéntes hálózatával közösen alig több mint 22 óra alatt [2]. A Triple-DES (3DES) ideiglenesen megoldotta a kulcsméret problémáját, de lassú volt, és 64 bites blokkmérete is korlátot jelentett, ezért új szabványra volt szükség.

Az *Advanced Encryption Standard* (AES) 2001-ben váltotta fel a DES-t. A NIST nyílt pályázatát a belga Joan Daemen és Vincent Rijmen Rijndael nevű algoritmusa nyerte meg [4]. Az AES SPN-szerkezetű, 128 bites blokkokkal dolgozik, kulcsa 128, 192 vagy 256 bites, amelyekhez rendre 10, 12 és 14 kör tartozik. Ma az AES a szimmetrikus kriptográfia etalonja: számos processzor hardveresen gyorsítja (pl. az Intel AES-NI utasításkészlettel), és a teljes algoritmus ellen nem ismert gyakorlatban kivitelezhető támadás. Ezért szolgál az AES-128 a dolgozatban viszonyítási alapként: azt vizsgáljuk, mennyi erőforrást takaríthatnak meg hozzá képest a könnyűsúlyú algoritmusok, és ennek milyen biztonsági ára van.

## 2.2. Hashfüggvények és üzenethitelesítés

A biztonságos kommunikációhoz a titkosság mellett *integritás* (az adat sértetlensége) és *hitelesség* (a feladó igazolása) is szükséges [1]. Ezeket kriptográfiai hashfüggvényekkel, üzenethitelesítő kódokkal (MAC) és hitelesített titkosítási sémákkal (AEAD) lehet biztosítani.

### 2.2.1. Kriptográfiai hashfüggvények

A hashfüggvény tetszőleges hosszúságú bemenetből rögzített méretű kimenetet, *lenyomatot* (digest) állít elő. Biztonsági célra akkor alkalmas, ha teljesíti a következő három feltételt [2]:

1. **Egyirányúság** (preimage resistance): a lenyomatból gyakorlatilag ne lehessen olyan bemenetet találni, amely ezt a lenyomatot adja.

2. **Második ősképpel szembeni ellenállás** (second preimage resistance): egy adott bemenethez ne lehessen olyan másik bemenetet találni, amelynek ugyanaz a lenyomata.

3. **Ütközésállóság** (collision resistance): ne lehessen hatékonyan találni két tetszőleges, különböző bemenetet azonos lenyomattal.

Ma a SHA-2 és a SHA-3 család a szabványos választás; az MD5 és a SHA-1 a megtalált ütközések miatt biztonsági célra már nem használható.

### 2.2.2. MAC – üzenethitelesítő kód

A MAC (Message Authentication Code) előállításához egy közös titkos kulcs is kell, ezért az integritás mellett a hitelességet is biztosítja: érvényes MAC-ot csak az tud előállítani, aki ismeri a kulcsot. A két legelterjedtebb változat a hashfüggvényre épülő HMAC [1] és a blokktitkosítót láncolva használó CBC-MAC.

### 2.2.3. AEAD – hitelesített titkosítás kapcsolódó adatokkal

A gyakorlat azt mutatta, hogy a titkosítást és a hitelesítést célszerű egyetlen, együtt tervezett eljárásban elvégezni. Ezt valósítják meg az AEAD (Authenticated Encryption with Associated Data) sémák [9]. Bemenetük a titkos kulcs, egy csak egyszer használható érték (nonce), a nyílt szöveg és a *kapcsolódó adat* (associated data, AD). Ez utóbbi, például egy hálózati csomag fejléce, titkosítatlan marad, hogy a hálózati eszközök olvashassák, de a séma hitelesíti, így módosítása kiderül.

A kimenet a titkosított szöveg és egy *hitelesítő címke* (tag), amely a titkosított szöveg és a kapcsolódó adat sértetlenségét igazolja. Az AEAD sémák egyik legfontosabb feltétele, hogy ugyanazzal a kulccsal egy nonce-ot ne használjunk kétszer, mert ez a legtöbb sémánál súlyosan sérti a biztonságot. A dolgozat szempontjából azért fontos az AEAD, mert az ASCON a többi vizsgált algoritmussal ellentétben nem blokktitkosító, hanem eleve ilyen sémának készült.

## 2.3. Erőforrás-korlátozott rendszerek jellemzői

A könnyűsúlyú kriptográfia célpontjai jellemzően kis teljesítményű, kevés memóriával és szűkös energiával rendelkező eszközök.

### 2.3.1. Mikrovezérlők és erőforrásaik

Az IoT-eszközökben használt processzorok szóméretük szerint három fő csoportba sorolhatók:

- **8 bites** (pl. ATmega): néhány MHz-es órajel, néhány kilobájt RAM. Egyszerű szenzorokban és beágyazott vezérlőkben gyakoriak.

– **16 bites** (pl. MSP430): nagyon alacsony fogyasztásra tervezett platformok, amelyeket gyakran használnak vezeték nélküli szenzorhálózatokban (WSN) [6].

– **32 bites** (pl. ARM Cortex-M, ESP32): ipari átjárókban és összetettebb szenzorrendszerekben használt eszközök, akár megabájtos flash-memóriával, de még így is nagyságrendekkel kisebb kapacitással, mint egy asztali számítógép.

Ezek a korlátok közvetlenül meghatározzák, milyen kriptográfiai megoldás futtatható az eszközön. A programkód és a konstansok a flash-memóriában, a változók és a körkulcsok a jellemzően sokkal kisebb RAM-ban kapnak helyet; egy RFID-címkén például mindössze 300–500 bájt RAM állhat rendelkezésre [8]. Sok kis processzorból hiányzik a hardveres szorzó vagy a kriptográfiai gyorsító, így minden műveletet egyszerű utasításokból kell felépíteni. Elemes eszközöknél minden elvégzett óraciklus energiát fogyaszt, ezért a kriptográfiai műveletek költsége is befolyásolja, meddig működik az eszköz egy töltéssel vagy elemcserével [6].

Ilyen eszközök működnek többek között az ipari rendszerekben (például PLC-k és SCADA-rendszerek környezetében) [25], az okosváros-infrastruktúrában és az egészségügyi eszközökben, ahol egyszerre van szükség erős adatvédelemre és takarékos működésre. Ezeken a területeken válik igazán fontossá a könnyűsúlyú kriptográfia.

## 2.4. A lightweight kriptográfia motivációja

### 2.4.1. A hagyományos szabványok korlátai

Az AES-t kifejezetten úgy tervezték, hogy sokféle platformon, a 8 bites smart cardoktól a szerverekig hatékonyan fusson [5]. A legszűkebb eszközökön azonban így is jelentős a költsége: a 256 bájtos S-box tábla a néhány kilobájtos memóriából érezhető részt foglal el, a véges test feletti MixColumns lépés pedig 8 bites processzoron sok utasítást igényel. Hardveres megvalósítása nagyjából 2400 kapuegyenértéknyi (gate equivalent, GE) területet foglal, ami a legolcsóbb RFID-chipeken túl sok [6].

A könnyűsúlyú kriptográfia ezért a szűkös erőforrások és az elvárt biztonság közötti egyensúlyt keresi. Egy IoT-eszköz általában nem igényel 256 bites biztonsági szintet; a 128 bites szint a jelenlegi ismeretek szerint az eszköz teljes élettartamára elegendő védelmet nyújt [7]. A cél tehát az, hogy ez a biztonsági szint a lehető legkevesebb memóriával és energiával legyen elérhető, például kisebb S-boxokkal vagy egyszerű ARX-műveletekkel.

### 2.4.2. A NIST Lightweight Cryptography szabványosítás

A NIST 2018-ban hirdetett nyílt pályázatot erőforrás-korlátozott eszközökre szánt hitelesített titkosítási (AEAD) sémák szabványosítására [10]. A jelölteknek legalább 128 bites biztonságot kellett nyújtaniuk, a meglévő NIST-szabványoknál lényegesen jobb hatékonyság mellett.

A beérkezett 57 pályázatból 56 jutott az első fordulóba, 2021-ben tíz döntős maradt, és 2023 februárjában a NIST az *ASCON*-t választotta ki. A döntésben szerepet játszott, hogy az ASCON egyetlen, 320 bites állapoton dolgozó permutációra épül, amelyet szivacs- (duplex-) konstrukcióban használ, így egyszerűen és hatékonyan megvalósítható hardverben és szoftverben is [9]. A végleges szabvány NIST SP 800-232 néven 2025-ben jelent meg [11].

# 3. fejezet

# Vizsgált algoritmusok bemutatása

Ez a fejezet a négy vizsgált algoritmus felépítését, kulcsgenerálását és azokat a tulajdonságait mutatja be, amelyekre a mérési eredmények értelmezésénél támaszkodunk.

## 3.1. AES-128

Az AES eredeti neve *Rijndael*, amely a két belga tervező, Joan Daemen és Vincent Rijmen nevéből állt össze. A NIST 1997-ben kiírt pályázatára 15 jelölt érkezett; a Rijndael a biztonsága, a hatékonysága és a sokféle platformon való jó megvalósíthatósága miatt nyert, és 2001-ben FIPS 197 néven szabvány lett [4, 5].

### 3.1.1. Felépítés és működés

Az AES-128 SPN-szerkezetű blokktitkosító: 128 bites blokkokat dolgoz fel 128 bites kulccsal, 10 körben. A belső állapot (*State*) egy $4 \times 4$-es bájtmátrix. Egy kör négy lépésből áll:

1. **SubBytes** – minden bájtot egy 256 elemű S-boxon vezet át. Az S-box a $GF(2^8)$ feletti multiplikatív inverzen és egy affin transzformáción alapul, ez adja a konfúziót.
2. **ShiftRows** – a mátrix $i$-edik sorát $i$ pozícióval balra forgatja, így egy oszlop bájtjai a következő lépésben különböző oszlopokba kerülnek.
3. **MixColumns** – minden oszlopot egy rögzített mátrixszal szoroz $GF(2^8)$ felett, így egy bemeneti bájt az oszlop mind a négy kimeneti bájtjára hat. Az utolsó körből ez a lépés kimarad.
4. **AddRoundKey** – XOR-ral hozzáadja az állapothoz az aktuális körkulcsot.

A visszafejtés ezek inverzeit alkalmazza fordított sorrendben. A titkosító MixColumns együtthatói kicsik (1, 2, 3), ezért a szorzás egy-egy kettővel való szorzásra (*xtime*) és XOR-ra vezethető vissza. Az inverz MixColumns együtthatói (9, 11, 13, 14) viszont nagyobbak, így szoftverben lényegesen több műveletet igényel. Ennek a mérésekben is jól látható következménye lesz (5.2. alfejezet).

### 3.1.2. Kulcsgenerálás

Az AES-128 kulcsgenerálása a 128 bites fő kulcsból 11 darab 128 bites körkulcsot (összesen 176 bájtot) állít elő; az első körkulcs maga a fő kulcs. Egy új körkulcs első oszlopához az előző körkulcs utolsó oszlopát egy bájttal elforgatjuk, bájtjait átvezetjük az S-boxon, hozzáadjuk a körkonstanst (Rcon), végül XOR-ral összekeverjük az előző körkulcs első oszlopával. A további oszlopok az előző oszlop és az előző körkulcs megfelelő oszlopának XOR-jaként adódnak [5].

### 3.1.3. Az AES-128 mint viszonyítási alap

Az AES-128 a dolgozatban nem könnyűsúlyú algoritmusként, hanem viszonyítási alapként szerepel, mert a legtöbb mai protokoll (TLS, VPN, banki rendszerek) erre épül. Korlátozott eszközökön a 256 bájtos S-box (és visszafejtéshez az inverze) tárolása, valamint a $GF(2^8)$ feletti aritmetika jelent többletköltséget. A vizsgált STM32L432KC-n nincs hardveres AES-gyorsító, így az algoritmus tisztán szoftveresen fut.

## 3.2. SPECK 128/128

A SPECK algoritmuscsaládot az amerikai Nemzetbiztonsági Ügynökség (NSA) tette közzé 2013-ban, a SIMON családdal együtt [12]. A SIMON elsősorban hardveres, a SPECK szoftveres megvalósításra készült. A családban több blokk- és kulcsméret is szerepel; mi a 128 bites blokkot és 128 bites kulcsot használó SPECK 128/128 változatot vizsgáljuk.

### 3.2.1. Körfüggvény

A SPECK 128/128 állapota két 64 bites szó ($x$, $y$). A titkosítás 32 körből áll, minden kör a következő ARX-lépést hajtja végre:

$$x \leftarrow ((x \ggg 8) \boxplus y) \oplus K_i, \tag{3.1}$$

$$y \leftarrow (y \lll 3) \oplus x, \tag{3.2}$$

ahol $\ggg$ és $\lll$ a jobbra, illetve balra forgatás, $\boxplus$ az összeadás $\bmod 2^{64}$, $K_i$ pedig az $i$-edik körkulcs. Egy kör tehát két forgatás, egy összeadás és két XOR; a nemlinearitást egyedül

a moduláris összeadás adja [12]. Visszafejtéskor ugyanezek a lépések fordított sorrendben, kivonással és ellentétes irányú forgatással futnak.

### 3.2.2. Kulcsgenerálás

A kulcsgenerálás ugyanazt a körfüggvényt használja, mint a titkosítás, csak a kör sorszáma tölti be a kulcs szerepét. A 128 bites kulcs két 64 bites szóra bomlik: $K = (l_0, k_0)$, ahol $k_0$ az első körkulcs. A további körkulcsok:

$$l_{i+1} = (k_i \boxplus (l_i \ggg 8)) \oplus i, \tag{3.3}$$

$$k_{i+1} = (k_i \lll 3) \oplus l_{i+1}. \tag{3.4}$$

Ennek két előnye van: a kulcsgeneráláshoz nem kell külön kód, és a körkulcsok menet közben (on the fly) is előállíthatók, nem kell mindet előre a RAM-ban tárolni.

### 3.2.3. Szoftveres megvalósítás

A SPECK nem használ táblát, végrehajtási ideje nem függ az adattól, és egy 64 bites processzoron minden művelete egyetlen utasítás. Egy 32 bites Cortex-M magon viszont a 64 bites szavakat regiszterpárokban kell kezelni: az összeadás két utasítás (ADDS és ADC), a 64 bites forgatás pedig több eltolásból és kombinálásból áll. Dinu és munkatársai összehasonlító mérésében a SPECK így is a leggyorsabb szoftveres könnyűsúlyú blokktitkosítók közé tartozott [17].

### 3.2.4. Bizalmi kérdések

A SPECK körüli viták elsősorban nem kriptoanalitikai, hanem bizalmi jellegűek. Az eredeti publikáció nem indokolta részletesen a tervezési döntéseket, például a forgatási paraméterek és a körszám megválasztását; ezt a tervezők 2017-ben részben pótolták [13]. A SIMON és a SPECK felvételét a könnyűsúlyú blokktitkosítókat tartalmazó ISO/IEC 29192-2 szabványba 2015-ben javasolták, de 2018 áprilisában elutasították, amiben a tervezővel szembeni bizalmatlanság is szerepet játszott [39]. Szűkebb területen, az RFID-eszközök légi interfészének kriptográfiai csomagjaként viszont mindkettő ISO/IEC-szabvány lett (ISO/IEC 29167-21 és 29167-22) [41]. A nyilvános kriptoanalízis ugyanakkor több mint egy évtized alatt sem talált a teljes körszámú változat ellen gyakorlatban kivitelezhető támadást (6.3. alfejezet).

## 3.3. LEA-128

A *Lightweight Encryption Algorithm* (LEA) a dél-koreai Nemzeti Biztonsági Kutatóintézet (NSRI) fejlesztése, amelyet Hong és munkatársai 2013-ban mutattak be [15]. Elsősorban általános célú, 32 bites processzorokon való gyors szoftveres futásra tervezték. 2019 óta az ISO/IEC 29192-2 szabvány része [16], Dél-Koreában pedig a KCMVP (Korean Cryptographic Module Validation Program) által jóváhagyott algoritmus.

### 3.3.1. Általánosított Feistel-hálózat

A LEA az ARX-műveleteket általánosított Feistel-hálózatban (Generalized Feistel Network, GFN) használja: a 128 bites blokkot négy 32 bites szóra bontja, $(X_0, X_1, X_2, X_3)$. Egy kör [15]:

$$X_0' = \mathrm{ROL}\big((X_0 \oplus RK_0) \boxplus (X_1 \oplus RK_1),\ 9\big), \tag{3.5}$$

$$X_1' = \mathrm{ROR}\big((X_1 \oplus RK_2) \boxplus (X_2 \oplus RK_3),\ 5\big), \tag{3.6}$$

$$X_2' = \mathrm{ROR}\big((X_2 \oplus RK_4) \boxplus (X_3 \oplus RK_5),\ 3\big), \tag{3.7}$$

$$X_3' = X_0, \tag{3.8}$$

ahol $RK_0, \ldots, RK_5$ a kör hat 32 bites körkulcsszava, $\boxplus$ az összeadás mod$2^{32}$, ROL és ROR pedig a balra, illetve jobbra forgatás. A LEA-128 24 körből áll, így összesen $24 \times$ $\times\, 6 = 144$ körkulcsszót használ.

### 3.3.2. A 32 bites szóméret

A LEA és a SPECK legfontosabb gyakorlati különbsége a szóméret. A LEA minden művelete 32 bites, így egy 32 bites magon mindegyik egyetlen utasítás, a forgatás pedig a Cortex-M4 eltolóegysége (barrel shifter) miatt akár egy másik utasításba is beolvasztható. A SPECK 128/128 64 bites szavait ugyanezen a magon csak több utasítással lehet kezelni. A tervezők ezért a LEA-t 32 bites platformokon a SPECK-nél gyorsabbnak várták [15]; hogy ez a mi méréseinkben mennyiben igazolódott, azt a 5. fejezet mutatja meg.

### 3.3.3. Kulcsgenerálás

A LEA-128 kulcsgenerálása négy 32 bites konstansra $(\delta_0, \ldots, \delta_3)$ épül, amelyek a $\sqrt{766965}$ hexadecimális kifejtéséből származnak (766965 a „LEA” betűinek ASCII-kódja). A fő kulcs négy szava $K_0, \ldots, K_3$; az $i$-edik körben

$$T_j = \mathrm{ROL}\big(T_j \boxplus \mathrm{ROL}(\delta_{i \bmod 4},\ i + j),\ r_j\big), \quad j = 0,1,2,3, \tag{3.9}$$

ahol kezdetben $T_j = K_j$, a forgatási paraméterek pedig $r_0 = 1$, $r_1 = 3$, $r_2 = 6$, $r_3 = 11$. A kör hat körkulcsszava $RK = (T_0, T_1, T_2, T_1, T_3, T_1)$. A kulcsgenerálás tehát szintén ARX-műveletekből áll, és menet közben is végrehajtható [15].

## 3.4. ASCON-128

Az ASCON-t a Grazi Műszaki Egyetem kutatói, Christoph Dobraunig, Maria Eichlseder, Florian Mendel és Martin Schläffer tervezték. 2014-ben a CAESAR hitelesített titkosítási versenyre nyújtották be, ahol 2019-ben a könnyűsúlyú kategória első számú választása lett, majd 2023-ban a NIST is kiválasztotta [9, 10]. A többi vizsgált algoritmussal ellentétben nem blokktitkosító, hanem egy permutációra épülő hitelesített titkosítási (AEAD) séma; ugyanerre a permutációra épül a család hashfüggvénye is.

### 3.4.1. Szivacs- és duplexkonstrukció

A szivacskonstrukció (sponge construction) egy rögzített permutációra épül, és a nagy belső állapotot két részre osztja: a *rate* ($r$ bit) részen keresztül olvassa be az adatot és adja ki az eredményt, a *capacity* ($c$ bit) rész kívülről közvetlenül soha nem érhető el. A konstrukció biztonsága a capacity méretén múlik. Az ASCON ennek kulcsolt, kétirányú változatát, a *duplexkonstrukciót* használja, amelyben minden adatblokk után lefut a permutáció, és a kimenet is a rate részből származik [9]. Az ASCON belső állapota 320 bites, öt 64 bites szóból áll $(x_0, \ldots, x_4)$.

### 3.4.2. Az ASCON-permutáció

Az ASCON alapeleme egy $p$ permutáció, amelyet az inicializáláskor és a véglegesítéskor $a = 12$, az adatfeldolgozás során $b$ körrel alkalmaznak. Egy kör három rétegből áll:

1. **Körkonstans hozzáadása**: egy körönként eltérő konstanst XOR-oz az $x_2$ szóhoz.
2. **Helyettesítő réteg**: 64 párhuzamos, 5 bites S-box. Az S-boxot nem táblázatként, hanem bitenkénti logikai műveletekkel (AND, XOR, NOT) számolják ki egyszerre mind az öt szón (*bitsliced* megvalósítás), így nincs memóriából olvasott tábla, és a végrehajtási idő nem függ az adattól.
3. **Lineáris diffúziós réteg**: minden szót önmaga két elforgatott változatával XOR-oz, például $x_0 \leftarrow x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28)$. A forgatási értékek szavanként eltérnek.

### 3.4.3. Az AEAD működése

Egy hitelesített titkosítás négy fázisból áll [9]:

1. **Inicializálás**: az állapot a kezdőérték (IV), a kulcs ($K$) és a nonce ($N$) összefűzése. Ezt 12 körös permutáció követi, majd a kulcsot XOR-ral az állapot végéhez adjuk.

2. **Kapcsolódó adat feldolgozása**: az AD-t $r$ bites blokkokra bontjuk, minden blokkot a rate részhez adjuk, és $b$ körös permutációt alkalmazunk. Üres AD esetén ez a lépés elmarad. Végül egyetlen bitet (domain separation) átfordítunk az állapotban, ami elválasztja a kapcsolódó adatot a nyílt szövegtől.

3. **Titkosítás**: a nyílt szöveg minden $r$ bites blokkját XOR-ral a rate részhez adjuk; az eredmény a titkosított blokk, és egyben a rate új értéke. Az utolsó, kiegészített blokk kivételével minden blokk után $b$ körös permutáció következik.

4. **Véglegesítés**: a kulcsot ismét az állapothoz adjuk, 12 körös permutációt alkalmazunk, végül az állapot utolsó 128 bitjét a kulccsal XOR-ozva kapjuk a hitelesítő címkét ($T$).

Visszafejtéskor a fogadó ugyanezt a folyamatot futtatja, kiszámolja a címkét, és összeveti a kapottal. Eltérés esetén a nyílt szöveget nem adja ki, így módosított vagy sérült üzenet nem kerül feldolgozásra.

Fontos következmény, hogy az üzenet hosszától függetlenül minden titkosítás tartalmaz két 12 körös permutációt. Ez üzenetenként fix költség, amely rövid üzeneteknél meghatározóvá válik (5.3. alfejezet).

### 3.4.4. A vizsgált változat

Az Ascon v1.2 specifikáció két AEAD-változatot ír le [9]. Az elsődlegesen ajánlott ASCON-128 rate-je 64 bit, capacity-je 256 bit, és adatfeldolgozáskor $b = 6$ kört használ. Az ASCON-128a rate-je 128 bit, capacity-je 192 bit, $b = 8$ körrel. A 128a bájtonként kevesebb permutációs kört futtat (16 bájtonként 8, szemben a 8 bájtonkénti 6 körrel), ezért hosszabb üzeneteken gyorsabb. A NIST SP 800-232 szabvány Ascon-AEAD128 sémája az ASCON-128a paramétereire épül, de kisebb módosításokkal (például más kezdőértékkel és little-endian bájtsorrenddel), így egyik v1.2-es változattal sem bitre azonos [11].

A dolgozatban az ASCON-128-at mértük. Ez a tervezők elsődleges ajánlása, a CAESAR-verseny végső portfóliójában is ez az elsődleges változat, és a hivatalos tesztvektorokkal (KAT) teljes körűen ellenőrizhető volt. A szabványos változat várható költségére a 5.3. alfejezetben adunk becslést.

## 3.5. Az algoritmusok összevetése

A 3.1. táblázat a négy algoritmus fő szerkezeti jellemzőit foglalja össze.

3.1. táblázat. A vizsgált algoritmusok szerkezeti jellemzői

| **Jellemző** | **AES-128** | **SPECK 128/128** | **LEA-128** | **ASCON-128** |
|---|---|---|---|---|
| Típus | blokktitkosító | blokktitkosító | blokktitkosító | AEAD |
| Szerkezet | SPN | ARX-Feistel | ARX-GFN | duplex (szivacs) |
| Blokkméret / rate | 128 bit | 128 bit | 128 bit | 64 bit (rate) |
| Kulcsméret | 128 bit | 128 bit | 128 bit | 128 bit |
| Körök száma | 10 | 32 | 24 | 12 / 6 |
| Szóméret | 8 bit (bájt) | 64 bit | 32 bit | 64 bit |
| S-box | 8 bites, 256 elem | – | – | 5 bites, bitsliced |
| Nemlinearitás | S-box | mod. összeadás | mod. összeadás | S-box (logikai) |
| Szabvány | FIPS 197 | ISO/IEC 29167-22† | ISO/IEC 29192-2 | SP 800-232* |

†Csak RFID-légi interfészhez; az általános ISO/IEC 29192-2-be nem került be.

*A szabványos Ascon-AEAD128 az ASCON-128a változatból készült (3.4.4. alfejezet).

Az AES a táblás S-boxra épülő SPN-t képviseli. A SPECK és a LEA az S-box nélküli, aritmetikai nemlinearitású ARX-irányzat két változata, amelyek szóméretben és ágszámban térnek el. Az ASCON a titkosítást és a hitelesítést egyetlen, permutációra épülő konstrukcióban valósítja meg. Mind a négy 128 bites kulcsot használ, ami azonos alapot ad a biztonsági szintek összevetéséhez. A három blokktitkosító 128 bites blokkmérete mellett a születésnapi korlát (6.2.5. alfejezet) $2^{64}$ blokk, ami IoT-alkalmazásokban nem jelent valódi korlátot.

# 4. fejezet

# Implementáció és mérési módszer

Ez a fejezet a mérési platformot, az algoritmusok megvalósítását és a közös mérőprogramot írja le, olyan részletességgel, hogy a mérések megismételhetők legyenek.

## 4.1. A mérési platform

A méréseket egy NUCLEO-L432KC fejlesztőkártyán végeztük. A kártyán lévő STM32L432KC mikrovezérlő ARM Cortex-M4 magos, legfeljebb 80 MHz-es órajellel, 256 KB flash- és 64 KB SRAM-memóriával. A Cortex-M4 32 bites mag: az összeadás, az XOR és a 32 bites forgatás egy-egy utasítás, 64 bites aritmetikája viszont nincs. A kártya beépített ST-LINK programozója a programletöltésre és a soros kommunikációra is szolgált.

A rendszerórajel 80 MHz, amelyet a belső MSI-oszcillátor (4 MHz) jeléből a PLL állít elő. Ezen a frekvencián a flash olvasása 4 várakozási ciklust (wait state) igényel. Ennek hatását az STM32L4 család ART-gyorsítója mérsékli, amely egy 1 KB-os utasítás-gyorsítótárból és egy 256 bájtos adat-gyorsítótárból áll, és mindkettő a flash-memória elé kapcsolódik [35]. A Cortex-M4 magnak saját gyorsítótára nincs, így az ART az egyetlen gyorsítótár a rendszerben. Mivel ezek a beállítások közvetlenül befolyásolják a mért ciklusszámokat, a 4.1. táblázat rögzíti őket.

4.1. táblázat. A mérési környezet

| **Elem** | **Érték** |
|---|---|
| Fejlesztőkártya | NUCLEO-L432KC |
| Mikrovezérlő | STM32L432KC (ARM Cortex-M4) |
| Órajel | 80 MHz (MSI 4 MHz → PLL), 1-es feszültségtartomány |
| Flash várakozási ciklusok | 4 (`FLASH_LATENCY_4`) |
| ART-gyorsító | utasítás- és adat-gyorsítótár be, előolvasás (prefetch) ki |
| Fejlesztőkörnyezet | STM32CubeIDE |
| Fordító | arm-none-eabi-gcc 14.3.1 (GNU Tools for STM32) |
| Fordítási kapcsolók | `-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -ffunction-sections -fdata-sections`, C11 |

## 4.2. A forráskód szerkezete

A mérőprogram egyetlen STM32CubeIDE-projekt. A hardver inicializálását (órajel, GPIO, USART2) a CubeMX által generált kód végzi, a saját kód a következő fájlokban van:

- `main.c` – önellenőrzés, mérési ciklus, statisztika és kiírás;
- `aes.c/h`, `speck.c/h`, `lea.c/h`, `ascon128.c/h` – az algoritmusok, mindegyik egy fájlpárban, a fájl elején a forrás megnevezésével;
- `build_opt.h` – a fordítási szint felismerése a GCC előre definiált makróiból (4.6. alfejezet).

Egyik algoritmus sem használ dinamikus memóriafoglalást: a körkulcsok és a kontextusok statikus vagy veremben lévő változókban kapnak helyet.

## 4.3. Az algoritmusok megvalósítása

Mind a négy algoritmus hordozható C-kód, assembly és platformspecifikus gyorsítás nélkül. A 4.2. táblázat mutatja, melyik kód mire épül, és hogyan ellenőriztük a helyességét. Hasonló összehasonlító méréseket korábban többek között Dinu és munkatársai [17], valamint Cazorla és munkatársai [18] végeztek; a mi vizsgálatunk ezekhez képest az ASCON-t és a fordítói optimalizáció hatását is magában foglalja.

4.2. táblázat. Az implementációk forrása és ellenőrzése

| **Algoritmus** | **Alap** | **Helyességellenőrzés** |
|---|---|---|
| AES-128 | `tiny-AES-c` [33] (The Unlicense), AES-128-ra szűkítve | NIST SP 800-38A, F.1.1 ECB-vektor [32] |
| SPECK 128/128 | NSA *Implementation Guide* v1.1, 6. fejezet [14] | az útmutató tesztvektora |
| LEA-128 | saját megvalósítás a specifikáció alapján [15,30] | KISA-tesztvektor [31] |
| ASCON-128 | Ascon v1.2 specifikáció [9] és a hivatalos `ascon-c` referenciakód [34] (CC0) | a hivatalos KAT-fájl mind az 1089 vektora (PC-n), ebből kettő a kártyán is |

### 4.3.1. AES-128

Az AES alapja a beágyazott projektekben gyakran használt `tiny-AES-c` könyvtár [33]. Ez nem használ előre kiszámolt nagy táblákat (T-táblákat), csak a 256 bájtos S-boxot és annak inverzét tárolja a flash-memóriában, a $GF(2^8)$-szorzást pedig futás közben számolja. A könyvtárat AES-128-ra szűkítettük, és az ECB- és a CTR-módot használtuk.

A megvalósítás két részlete magyarázza az AES méréseit. A MixColumns oszloponként 4 `xtime` hívással (kettővel való szorzással) dolgozik. Az inverz MixColumns viszont egy általános `Multiply` makróval szoroz, amely az együtthatótól függetlenül mindig kiszámolja a teljes, 10 `xtime` hívásból álló láncot, és oszloponként 16 ilyen szorzást végez. Egy blokk titkosítása így 144, visszafejtése 5760 `xtime` hívást tartalmaz, ami negyvenszeres különbség. A makró elágazás nélküli, ezért a visszafejtés ideje sem függ az adattól, de jóval drágább.

Fontos, hogy a `tiny-AES-c` méretre és hordozhatóságra, nem sebességre készült. Az AES-re vonatkozó eredményeink erre az implementációra érvényesek; Cortex-M-re optimalizált (például T-táblás vagy bitsliced) megvalósítások lényegesen gyorsabbak lehetnek.

### 4.3.2. SPECK 128/128

A SPECK-hez az NSA nyilvános megvalósítási útmutatójának referenciakódját használtuk [14]. A körfüggvényt két makró valósítja meg (`ER64` titkosításhoz, `DR64` visszafejtéshez), a kulcsgenerálás ugyanazt az `ER64`-et hívja a kör sorszámával. A 32 körkulcsot (256 bájt) előre kiszámoljuk. A függvények 64 bites szavakkal dolgoznak, ezért a puffereket 8 bájtos határra igazítottuk, és little-endian bájtsorrendet használunk, ahogy az útmutató is.

### 4.3.3. LEA-128

A LEA-t a specifikáció [15] és Lee és munkatársai leírása [30] alapján saját magunk írtuk meg C-ben, mert a KISA referenciakönyvtára egy sokfájlos, több platformra és üzemmódra felkészített csomag, amelyből a mérési célhoz csak a blokkművelet kellett. A blokkot négy 32 bites szóként, little-endian sorrendben olvassuk be (a Cortex-M4-en ez egy-egy `LDR` utasítás). A specifikáció szerinti szócserét ($X_3' = X_0$) nem másolással, hanem a szavak szerepének körönkénti forgatásával oldottuk meg, így egy körben csak a három új szó kiszámítása történik. A kulcsgenerálás a 144 körkulcsszót előre kiszámolja. A kód a 192 és 256 bites kulcsot is kezeli, de a mérésekben csak a 128 bites változatot használtuk.

### 4.3.4. ASCON-128

Az ASCON-128-at az Ascon v1.2 specifikáció [9] és a tervezők hivatalos referenciakódja (`ascon-c`, `ascon128v12/ref`) [34] alapján egyetlen fájlpárba írtuk meg. Az állapot öt 64 bites szó; a permutáció egy körét egy `ROUND` függvény, a 12 és 6 körös változatot a `P12` és `P6` függvény valósítja meg. Az inicializálás, a kapcsolódó adat feldolgozása és a véglegesítés külön segédfüggvény, a bájtsorrend a specifikációnak megfelelően big-endian. Visszafejtéskor a címkét konstans időben, korai kilépés nélkül hasonlítjuk össze. A helyességet PC-n a hivatalos KAT-fájl mind az 1089 vektorával ellenőriztük; ezek közül kettőt a mikrovezérlő is lefuttat induláskor.

## 4.4. A mérési keretrendszer

### 4.4.1. Önellenőrzés és bemenetek

A program induláskor minden algoritmusra lefuttatja a 4.2. táblázatban felsorolt tesztvektorokat, majd visszafejti az eredményt, és összeveti az eredeti nyílt szöveggel. Az ASCON-nál azt is ellenőrzi, hogy egy bitjében módosított titkosított szöveget a visszafejtés elutasít. Ha bármelyik ellenőrzés hibát jelez, a mérés el sem indul.

A mérésekben minden algoritmus ugyanazt a 128 bites kulcsot kapja (hexadecimálisan `0123456789ABCDEF`, kétszer egymás után), a nyílt szöveg $i$-edik bájtja $i$ mod 256. Az üzenetméretek 16, 64, 256 és 1024 bájt. Az ASCON nonce-a csupa nulla, a kapcsolódó adat üres.

### 4.4.2. Ciklusmérés a DWT-egységgel

A Cortex-M4 hibakereső egysége, a DWT (Data Watchpoint and Trace) egy 32 bites ciklusszámlálót (`CYCCNT`) tartalmaz, amely minden órajelciklusban eggyel nő [36]. A `DWT_-`

`Init()` engedélyezi a nyomkövetést és elindítja a számlálót, egy művelet mérését pedig a `MEASURE` makró végzi (4.1. kódrészlet).

```
void DWT_Init(void) {
    DEMCR |= DEMCR_TRCENA;          /* nyomkovetes engedelyezese */
    DWT_CYCCNT = 0;
    DWT_CTRL |= DWT_CR_CYCCNTENA;   /* szamlalo inditasa */
}

#define MEASURE(ALGO, OP, SIZE, CALL)                          \
    do {                                                       \
        CALL;                                /* bemelegites */ \
        __disable_irq();                                       \
        uint32_t t0_ = DWT_CYCCNT;                             \
        for (int k_ = 0; k_ < ITERATIONS; k_++) { CALL; }      \
        uint32_t t1_ = DWT_CYCCNT;                             \
        __enable_irq();                                        \
        Record_Result((ALGO), (OP), (SIZE), (t1_ - t0_) / ITERATIONS); \
    } while (0)
```

4.1. Kódrészlet. A ciklusszámláló indítása és egy művelet mérése

Egy mérés menete: egy nem mért, bemelegítő hívás (ez tölti fel az ART gyorsítótárait), majd tiltott megszakítások mellett a művelet 100-szori ismétlése, végül az egy hívásra jutó átlag. A megszakítások tiltása kizárja, hogy a HAL 1 ms-os SysTick-megszakítása belekerüljön a mért időbe. A mért érték tartalmazza a függvényhívás és a blokkokon végigmenő ciklus költségét is. Ezt külön nem mértük; néhány tíz ciklusra becsülve a legrövidebb, kb. 800 ciklusos műveletnél is legfeljebb néhány százalék, a hosszabb üzeneteknél elhanyagolható.

A 32 bites számláló 80 MHz-en kb. 53,7 másodpercenként fordul át. Az előjel nélküli kivonás egy átfordulást helyesen kezel; a leghosszabb mért szakasz (az optimalizálás nélküli AES-visszafejtés 100-szor 1024 bájton) kb. 22 másodperc, jóval a korlát alatt.

### 4.4.3. Ismétlések és statisztika

A teljes mérési kört (kulcsgenerálás, majd titkosítás és visszafejtés mind a négy méreten) a program tízszer egymás után lefuttatja. Minden mért értékre a tíz kör mediánját, minimumát és maximumát, valamint a $(\max - \min)/\mathrm{medin}$ relatív eltérést számolja. A 5. fejezetben minden szám a medián.

### 4.4.4. Az eredmények kiírása

A mikrovezérlő az eredményeket az USART2 perifériáján (115 200 baud, az ST-LINK virtuális soros portja) küldi a számítógépre. A `printf()` kimenetét a `_write()` függvény átírásával a `HAL_UART_Transmit()` hívásra irányítottuk. A kiírás mindig a mért szakaszokon kívül történik: a program körönként CSV-sorokat (futás sorszáma, algoritmus, művelet, üzenetméret, ciklusszám), a végén pedig összesítő táblázatot küld. A naplókat a `graphics.py` Python-szkript (pandas, matplotlib) dolgozza fel, ez készíti a táblázatok adatait és az ábrákat is.

## 4.5. Az üzemmódok megválasztása

A három blokktitkosítót ECB módban mértük. Az ECB a blokkműveleten felül semmilyen láncolást vagy számlálókezelést nem végez, így a mért költség csak a blokktitkosító sajátja. Biztonsági okból (2.1.2. alfejezet) éles rendszerben nem használható, itt kizárólag mérési célt szolgál.

Az AES-nél kiegészítésként a `tiny-AES-c` CTR-módját is mértük. CTR módban a visszafejtés ugyanazt a titkosító függvényt hívja, mint a titkosítás, így ez a mérés megmutatja, hogy az ECB-módú visszafejtés többletköltsége valóban az inverz műveletekből ered-e.

Az ASCON-128-at natív AEAD-módjában mértük, üres kapcsolódó adattal. Költsége így a hitelesítő címke kiszámítását (visszafejtéskor ellenőrzését) is tartalmazza, amit a blokktitkosítókkal való összevetéskor figyelembe kell venni.

## 4.6. Fordítási beállítások

A teljes programot hat optimalizációs szinttel fordítottuk és mértük le (4.3. táblázat) [37]. A szintet mindig a projekt egészére állítottuk, hogy minden forrásfájl azonos beállítással forduljon. Ezt a program futás közben ellenőrzi is: minden algoritmusfájl a `build_opt.h` segítségével visszaadja a saját fordítási szintjét, és eltérés esetén a mérés nem indul el.

Az `-Os` és az `-Oz` a vizsgált kódra Cortex-M4-en bájtra azonos gépi kódot eredményezett (ezt a lefordított objektumfájlok összevetésével ellenőriztük), és a mért ciklusszámok is minden mérési pontban megegyeztek. A továbbiakban ezért a kettőt együtt, `-Os/-Oz` jelöléssel szerepeltetjük.

4.3. táblázat. A mért optimalizációs szintek

| **Kapcsoló** | **Jelentés** |
|---|---|
| `-O0` | optimalizálás nélkül (a fejlesztés alapbeállítása) |
| `-O2` | általános sebességoptimalizálás; a GCC leírása szerint a méret és a sebesség közötti kompromisszumot igénylő lépések nélkül |
| `-O3` | erős sebességoptimalizálás, többek között agresszívebb beágyazással és ciklusbontással |
| `-Ofast` | `-O3`, a lebegőpontos szabványmegfelelés lazításával |
| `-Os` | kódméret-optimalizálás |
| `-Oz` | erős kódméret-optimalizálás |

# 5. fejezet

# Teljesítménymérések és eredmények

Ez a fejezet a 4. fejezetben leírt mérőprogrammal kapott eredményeket mutatja be. Minden érték tíz mérési kör mediánja. A mérések gyakorlatilag determinisztikusak: a körök közötti legnagyobb relatív eltérés az összes mérési pontot tekintve 0,022 % volt, a legtöbb pontban nulla. Ennek oka, hogy a mért kód futása nem függ az adattól, megszakítás nem zavarja, és a bemelegítés után a gyorsítótár állapota is minden ismétlésnél azonos. Ez kontrollált benchmarkkörnyezet: valós alkalmazásban a megszakítások és a hideg gyorsítótár a mértnél nagyobb költséget is okozhatnak. Az ábrákon a SPECK-128 a SPECK 128/128-at jelöli.

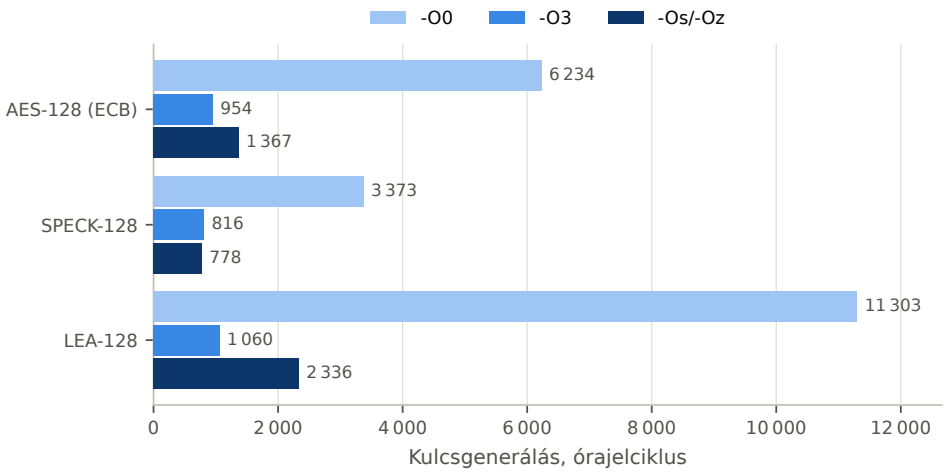
## 5.1. A kulcsgenerálás költsége

A 5.1. táblázat a kulcsgenerálás ciklusszámát mutatja minden szinten, a 1. ábra pedig három jellemző szintet.

5.1. táblázat. A kulcsgenerálás ciklusszáma optimalizációs szintenként

| **Algoritmus** | `-O0` | `-O2` | `-O3` | `-Ofast` | `-Os/-Oz` |
|---|---|---|---|---|---|
| AES-128 | 6 234 | 1 148 | 954 | 972 | 1 367 |
| SPECK-128 | 3 373 | 816 | 816 | 816 | 778 |
| LEA-128 | 11 303 | 1 060 | 1 060 | 1 060 | 2 336 |

Az `-O0`-ról `-O3`-ra váltva a kulcsgenerálás az AES-nél 6,5-szer, a SPECK-nél 4,1-szer, a LEA-nál 10,7-szer gyorsult. Optimalizálás nélkül a LEA a legdrágább, mert körönként hat körkulcsszót kell előállítania, és a sok kis segédfüggvény (forgatás, konstansszámítás) mind külön hívás. Sebességre optimalizált kódban (`-O2`, `-O3`, `-Ofast`) a három algoritmus közel azonos, 800–1150 ciklus. Az `-Os/-Oz` szinten a LEA kulcsgenerálása ismét kétszer drágább, mert a fordító a konstansszámító segédfüggvényt nem ágyazza be; a SPECK-é viszont itt a leggyorsabb (778 ciklus).

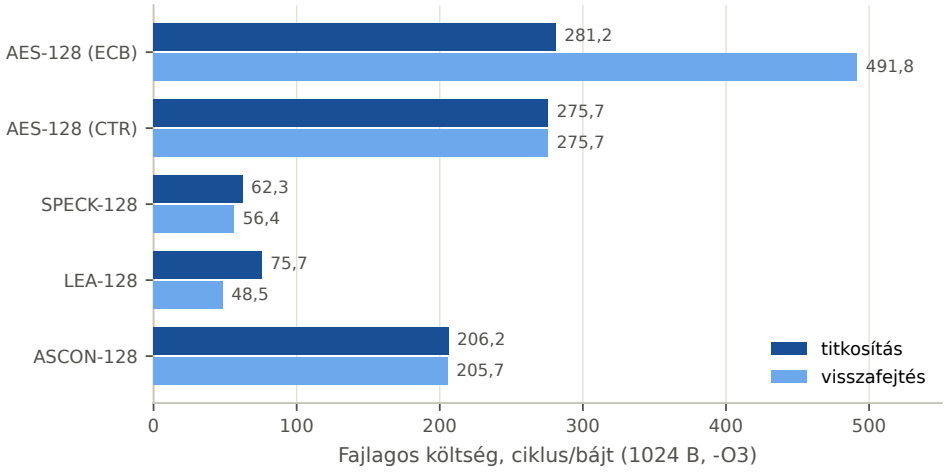



1. ábra. A kulcsgenerálás ciklusszáma `-O0`, `-O3` és `-Os/-Oz` mellett. Az ASCON-128 nem szerepel, mert nincs külön kulcsgenerálási lépése.

Az ASCON-128 nem használ külön kulcskiterjesztést: a kulcsot az inicializáláskor és a véglegesítéskor keveri be (3.4.3. alfejezet), így ez a költség a titkosítás mért idejében jelenik meg. Gyakorlati szempontból a kulcsgenerálás akkor számít, ha a kulcs gyakran változik: `-O3` mellett a LEA kulcsgenerálása nagyjából egy blokk titkosításának, az AES-é egy blokk ötödének felel meg.

## 5.2. Titkosítás és visszafejtés

A 2. ábra a fajlagos költséget (ciklus/bájt, cycles per byte, CPB) mutatja 1024 bájtos üzeneten `-O3` mellett. A többi szint értékei a 5. ábrán láthatók.




2. ábra. Fajlagos költség 1024 bájtos üzeneten, `-O3` mellett

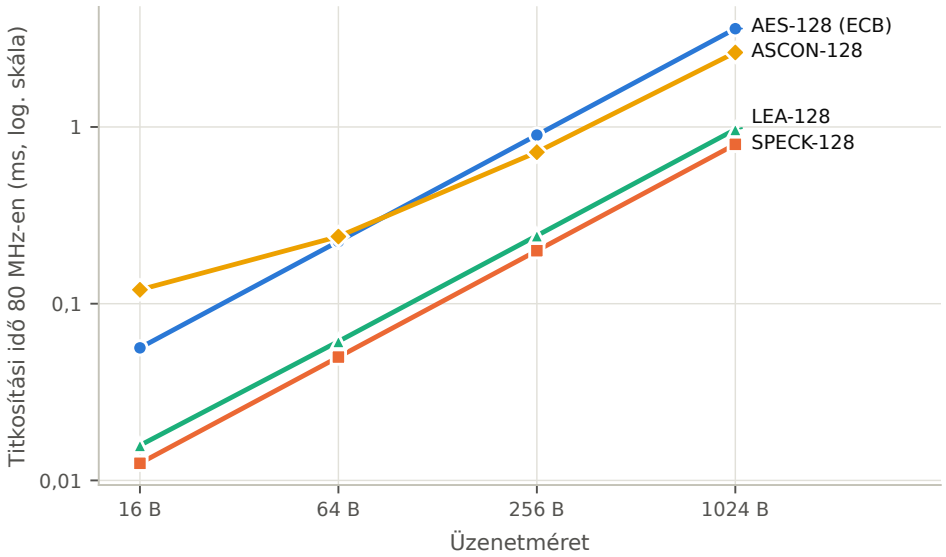
Az AES ECB módban erősen aszimmetrikus: a visszafejtés `-O0` mellett 9,1-szer, `-O3`

mellett 1,75-ször drágább a titkosításnál. Ezt az inverz MixColumns magyarázza, amely a használt implementációban blokkonként negyvenszer annyi `xtime` hívást tartalmaz, mint a titkosító irány (4.3. alfejezet). Optimalizálás nélkül minden ilyen hívás valódi függvényhívás, ezért a különbség óriási. `-O3` mellett a fordító beágyazza a függvényt, és mivel a szorzó együtthatók konstansok, a felesleges tagok és az ismétlődő számítások nagy részét kiejtheti, így az aszimmetria jóval kisebb lesz. CTR módban a két irány költsége minden szinten azonos (`-O3` mellett 275,7 CPB), mert a visszafejtés is a titkosító függvényt hívja. Az aszimmetria tehát az inverz blokkművelet költsége, amelyet megfelelő üzemmóddal el lehet kerülni.

Az ASCON-128 két iránya szintén azonos költségű (`-O3` mellett 206,2 és 205,7 CPB), mert a duplexkonstrukcióban mindkét irány ugyanazt a permutációt futtatja. A SPECK visszafejtése minden szinten kb. 10 %-kal gyorsabb a titkosításnál. A LEA-nál `-O0` mellett a két irány gyakorlatilag azonos, `-O3` mellett viszont a visszafejtés (48,5 CPB) jóval gyorsabb a titkosításnál (75,7 CPB), és ezzel a teljes mérés leggyorsabb művelete. Mivel a különbség csak optimalizált kódban jelentkezik, és szintenként változik (5.6. alfejezet), valószínűleg a fordító által generált kódból ered, nem magából az algoritmusból.

## 5.3. Skálázódás az üzenetmérettel

A 3. ábra a titkosítás idejét mutatja az üzenetméret függvényében, a 5.2. táblázat pedig a mért ciklusszámokat, mindkettő `-O3` mellett.




3. ábra. A titkosítás ideje az üzenetméret függvényében (`-O3`, 80 MHz, mindkét tengely logaritmikus)

5.2. táblázat. A titkosítás ciklusszáma az üzenetméret függvényében (`-O3`)

| **Algoritmus** | **16 B** | **64 B** | **256 B** | **1024 B** |
|---|---|---|---|---|
| AES-128 (ECB) | 4 500 | 18 002 | 71 988 | 287 937 |
| AES-128 (CTR) | 4 508 | 17 754 | 70 752 | 282 362 |
| SPECK-128 | 1 000 | 3 991 | 15 955 | 63 811 |
| LEA-128 | 1 263 | 4 893 | 19 413 | 77 493 |
| ASCON-128 | 9 592 | 19 192 | 57 592 | 211 192 |

A blokktitkosítók költsége egyenesen arányos az üzenet hosszával: ECB módban minden 16 bájtos blokk ugyanannyiba kerül, így a CPB minden méreten gyakorlatilag állandó (az AES-é 16 bájton 281,3, 1024 bájton 281,2). A logaritmikus ábrán ezért párhuzamos egyeneseket látunk.

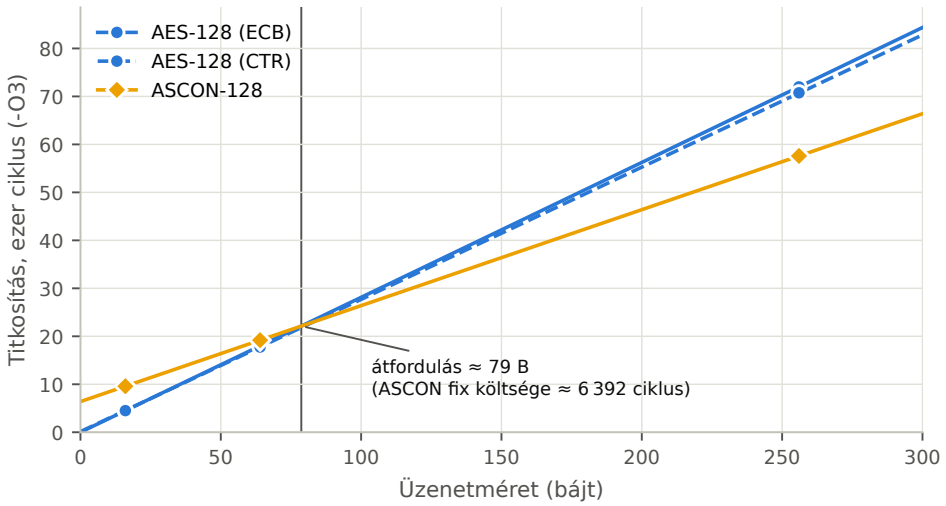
Az ASCON-128 viselkedése eltér ettől: CPB-je 16 bájton 600, 1024 bájton 206. A négy mérési pont `-O3` mellett pontosan illeszkedik a

$$C_{\mathrm{ASCON}}(n) \approx 6\,392 + 200 \cdot n \tag{5.1}$$

egyenesre, ahol $n$ az üzenet hossza bájtban. Mindkét tag visszavezethető a felépítésre. A meredekség azt jelenti, hogy egy 8 bájtos blokk 1600 ciklusba kerül, és mivel minden blokkhoz egy 6 körös permutáció tartozik, egy permutációs kör kb. 267 ciklus. A tengelymetszet ennek nagyjából 24-szerese, ami pontosan az inicializálás és a véglegesítés két 12 körös permutációja. Ellenőrzésképpen: 16 bájtnál két adatblokk és a két fix permutáció összesen 36 kört jelent, ami $36 \times 267 \approx 9\,600$ ciklus, a mért érték 9 592.

Ebből következik, hogy az ASCON-128 rövid üzeneteken lassabb az AES-nél, hosszabbakon gyorsabb (4. ábra). Az összevetés nem azonos szolgáltatások között történik: az ASCON a hitelesítő címkét is kiszámítja, az ECB és a CTR módú AES nem. A két egyenes `-O3` mellett kb. 79 bájtnál metszi egymást (CTR módú AES-sel kb. 83 bájtnál). Az átfordulási pont erősen függ a fordítási szinttől: `-O2` mellett kb. 54 bájt, `-Os`/`-Oz` mellett, ahol az ASCON a leggyorsabb, az AES pedig lassú, már kb. 23 bájt. Egy hitelesítést is nyújtó AES-alapú séma ennél drágább lenne: CCM módban például blokkonként két AES-hívás kell (egy a CBC-MAC-hez, egy a CTR-titkosításhoz), így ott az átfordulási pont jóval alacsonyabbra kerülne. Ezt nem mértük.

A szabványos Ascon-AEAD128 költségére ebből becslést is adhatunk. Az ASCON-128a paramétereivel 16 bájtonként 8 kör fut, vagyis bájtonként 0,5 kör a 0,75 helyett. Változatlan körköltséggel számolva a meredekség kb. 133 ciklus/bájtra csökkenne, a fix költség nem változna, és az AES-szel való átfordulás `-O3` mellett kb. 43 bájtra kerülne. Ez csak becslés, amelyet méréssel kell ellenőrizni (7.5. alfejezet).




4. ábra. Az ASCON-128 és az AES-128 titkosítási költsége rövid üzeneteken (`-O3`); a vonalak a mérési pontokra illesztett egyenesek

## 5.4. Feldolgozási sebesség

A feldolgozási sebesség (throughput) azt mutatja, hány bájtot tud a mikrovezérlő másodpercenként titkosítani vagy visszafejteni; ez nem azonos a kommunikációs csatorna átviteli sebességével. Az órajelből és a fajlagos költségből adódik: $v = f_{\mathrm{CLK}}/\mathrm{CPB}$, ahol 1 kB = 1000 B. Az 1024 bájtos üzenetre vonatkozó értékeket a 5.3. táblázat mutatja a gyakorlatban leginkább használt három szinten.

5.3. táblázat. Feldolgozási sebesség 1024 bájtos üzeneten, 80 MHz-en (kB/s; T: titkosítás, V: visszafejtés)

| **Algoritmus** | `-O2` | | `-O3` | | `-Os/-Oz` | |
|---|---|---|---|---|---|---|
| | T | V | T | V | T | V |
| AES-128 (ECB) | 220 | 129 | 285 | 163 | 179 | 103 |
| AES-128 (CTR) | 214 | 214 | 290 | 290 | 172 | 172 |
| SPECK-128 | 1 284 | 1 419 | 1 284 | 1 419 | 1 283 | 1 417 |
| LEA-128 | 1 135 | 1 581 | 1 057 | 1 650 | 1 387 | 1 484 |
| ASCON-128 | 340 | 334 | 388 | 389 | 412 | 413 |

A legnagyobb feldolgozási sebességet a LEA visszafejtése érte el `-O3` mellett (1 650 kB/s). Titkosításban a legjobb érték a LEA-é `-Os/-Oz` mellett (1 387 kB/s), a SPECK pedig minden optimalizált szinten 1 284 kB/s-ot ért el. Ugyanazon a szinten (`-O3`) a leglassabb művelet, az AES ECB-visszafejtése és a leggyorsabb között tízszeres a különbség.

Az ASCON-128 `-O3` mellett kb. 1,3-szer, `-Os/-Oz` mellett 2,4-szer gyorsabb a CTR

módú AES-nél, miközben a hitelesítő címkét is kiszámítja. A csak titkosságot nyújtó SPECK-nél és LEA-nál viszont 2,7–4,2-szer lassabb.

## 5.5. Kódméret és memóriaigény

A kódméretet és a veremigényt a mérőprogrammal azonos kapcsolókkal végzett külön fordításból határoztuk meg. A flash-igény a linker térképfájljából (`.map`) adódik, a nem használt függvényeket eltávolító `-gc-sections` linkelés után, a veremigény pedig a GCC `-fstack-usage` és `-fcallgraph-info` kapcsolójának kimenetéből, a hívási lánc mentén összegezve. Ehhez a fordításhoz az arm-none-eabi-gcc 13.3.1-es verzióját használtuk, ezért a bájtértékek a mérésnél használt 14.3.1-es fordítóétól kis mértékben eltérhetnek, az arányok azonban nem. Az eredményeket a 5.4. táblázat foglalja össze.

5.4. táblázat. Flash-igény (kód és konstansok) és legnagyobb veremhasználat bájtban, a mért függvényekre

| **Algoritmus** | **Flash (B)** | | | | **Verem (B)** | | | |
|---|---|---|---|---|---|---|---|---|
| | `-O0` | `-O2` | `-O3` | `-Os` | `-O0` | `-O2` | `-O3` | `-Os` |
| AES-128* | 4 135 | 1 995 | 3 995 | 1 541 | 120 | 88 | 176 | 108 |
| SPECK-128 | 778 | 292 | 292 | 280 | 88 | 32 | 32 | 20 |
| LEA-128† | 3 168 | 1 508 | 1 508 | 1 358 | 152 | 88 | 88 | 88 |
| ASCON-128 | 3 202 | 22 784 | 27 208 | 1 768 | 688 | 256 | 232 | 240 |

Az `-Ofast` értékei az `-O3`-éval, az `-Oz` értékei az `-Os`-éval azonosak. A veremigény a könyvtári `memcpy` nélkül értendő.

*Az ECB- és a CTR-módot, valamint a $2 \times 256$ bájtos S-box-táblákat is tartalmazza.

†A 192 és 256 bites kulcs kezelését is tartalmazza.

A kulcsfüggő adatok a RAM-ban külön helyet foglalnak: az AES kontextusa 192 bájt (176 bájt körkulcs és a CTR-mód 16 bájtos számlálója), a SPECK 32 körkulcsa 256 bájt, a LEA kulcsstruktúrája 772 bájt, amelyből 128 bites kulcsnál 576 bájt van használatban. Az ASCON-nak nincs tárolt körkulcsa, a 40 bájtos állapot a veremben van, és a táblázat veremértéke már tartalmazza. A mérőprogram saját puffereit ($3 \times 1040$ bájt) nem számoltuk az algoritmusokhoz, mert ezek az alkalmazástól függnek.

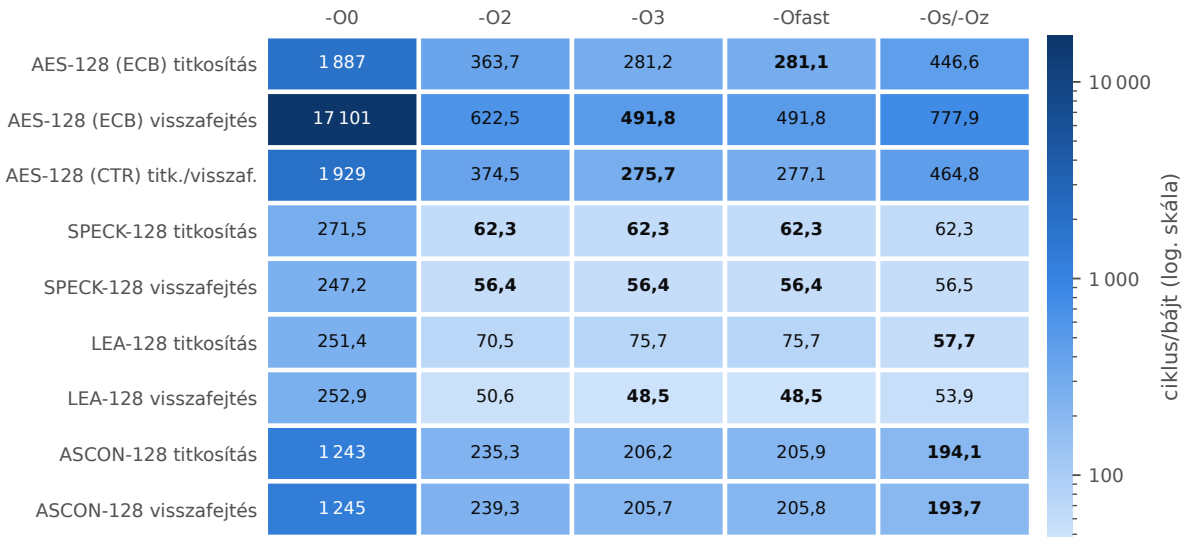
A legfeltűnőbb az ASCON: `-Os` mellett 1,8 KB, `-O2` és `-O3` mellett viszont 22–27 KB. Ennek oka, hogy a forrásban a `ROUND`, a `P6` és a `P12` `static inline` függvény, ezért sebességre optimalizálva a fordító a köröket kibontja, és a 12, illetve 6 körös permutációt minden hívási helyre külön beágyazza. Méretre optimalizálva ezt nem teszi meg. Csak a kapcsolódó adatot feldolgozó segédfüggvény, amely két 6 körös permutációt tartalmaz, 4,5 KB; egyetlen kibontott 6 körös permutáció tehát kb. 2 KB, ami jóval nagyobb az

ART 1 KB-os utasítás-gyorsítótáránál. `-Os` mellett a `ROUND` függvény 394 bájt, és egy ciklus hívja, így a teljes forró kódrész elfér a gyorsítótárban. Ennek a sebességre gyakorolt hatását a következő alfejezet tárgyalja.

A SPECK kódja minden szinten néhány száz bájt, mert egy körfüggvényből áll, amelyet a kulcsgenerálás is használ. 8 és 16 bites eszközökön, ahol a flash néhány tíz kilobájt (2.3.1. alfejezet), a kódméret önmagában is kizárhat egy változatot; az ASCON-t ilyen eszközre méretre optimalizálva érdemes fordítani.

## 5.6. A fordítói optimalizáció hatása

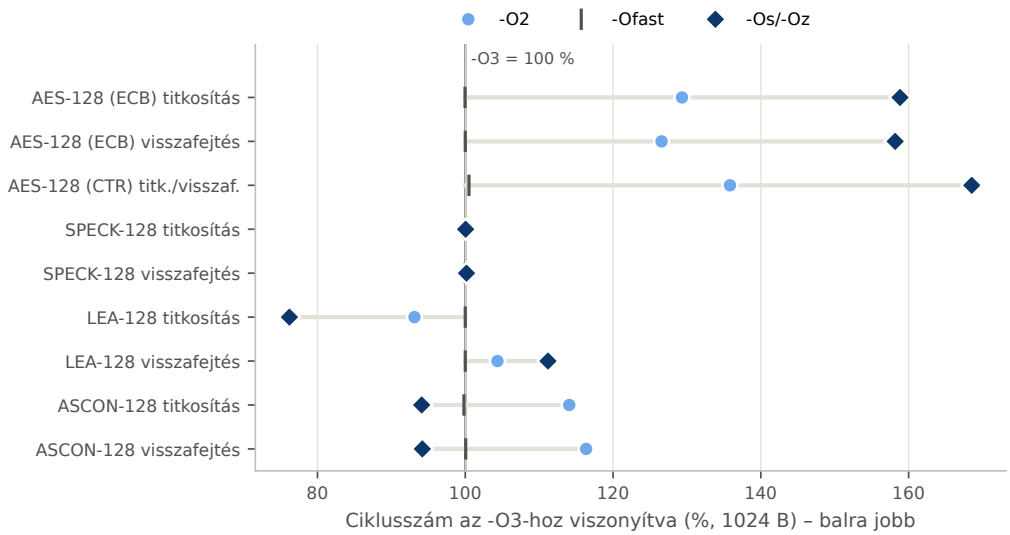
A 5. ábra minden algoritmus és minden szint fajlagos költségét mutatja, a 6. ábra pedig az `-O3`-hoz viszonyított eltérést.




5. ábra. Fajlagos költség (ciklus/bájt) 1024 bájtos üzeneten, minden optimalizációs szinten. Soronként félkövér a legjobb érték.

### 5.6.1. Optimalizálás nélkül és optimalizálva

Az optimalizálás minden algoritmust jelentősen gyorsított: 1024 bájtos titkosításnál `-O0`-hoz képest `-O3` mellett az AES 6,7-szer, az ASCON 6,0-szor, a SPECK 4,4-szer, a LEA 3,3-szer gyorsabb. A legnagyobb a különbség az AES ECB-visszafejtésénél: 17,5 millióról 504 ezer ciklusra, azaz 34,8-szeresére gyorsult, ami az előző alfejezetekben leírt `xtime`-hívásokkal magyarázható. Az `-O0` eredményei ezért elsősorban azt mutatják, hogy fejlesztői beállítással mért adatokból nem szabad következtetni az algoritmusok valódi költségére.




6. ábra. A ciklusszám az `-O3`-on mért érték százalékában (1024 B); 100 % alatt az adott szint gyorsabb

### 5.6.2. `-O2`, `-O3` és `-Ofast`

A SPECK már `-O2` mellett eléri a legjobb értékét, az `-O3` és az `-Ofast` sem javít rajta, mert a rövid, egyszerű körfüggvényen nincs mit tovább optimalizálni. Az AES és az ASCON `-O3` mellett 12–26 %-kal kevesebb ciklust igényel, mint `-O2` mellett, a LEA titkosítása viszont `-O2` mellett 7 %-kal gyorsabb. Az `-Ofast` sehol nem tért el 0,5 %-nál jobban az `-O3`-tól. Ez várható volt, mert az `-Ofast` többlete a lebegőpontos számításokat érinti, a vizsgált algoritmusok pedig csak egész műveleteket használnak.

### 5.6.3. `-Os/-Oz`

A méretre optimalizált kód hatása algoritmusonként eltér. Az AES ECB módban kb. 58 %-kal, CTR módban 69 %-kal lassabb, mint `-O3` mellett, mert a fordító a sok kis segédfüggvényt (`xtime`, `AddRoundKey`) nem ágyazza be. A SPECK gyakorlatilag érzéketlen a beállításra (0,2 % alatti eltérés). A LEA titkosítása 23,8 %-kal gyorsabb, visszafejtése 11,2 %-kal lassabb. Az ASCON mindkét irányban kb. 6 %-kal gyorsabb, mint `-O3` mellett, és 18 %-kal, mint `-O2` mellett; ezen a szinten éri el a legjobb eredményét.

Az ASCON viselkedésére a kódméret ad magyarázatot (5.5. alfejezet). `-O2` és `-O3` mellett a kibontott permutáció nem fér el az ART 1 KB-os utasítás-gyorsítótárában, ezért minden körben újra a flash-ből kell olvasni, 4 várakozási ciklussal, előolvasás nélkül. A kevesebb utasításból álló, de gyorsítótárba férő `-Os` kód ezt elkerüli. Ezzel összhangban a két nagy kódot adó szint (`-O2` és `-O3`) egyaránt lassabb az `-Os`-nél, pedig több optimalizálást végez. Teljes bizonyításhoz a mérést kikapcsolt utasítás-gyorsítótárral is meg kellene ismételni.

A LEA-nál ez a magyarázat nem elégséges: a kód mindhárom optimalizált szinten 1,3–1,5 KB, és a forró ciklus mindegyiknél elfér a gyorsítótárban. Itt valószínűleg a generált utasítássorrend és a regiszterkiosztás különbsége okozza az eltérést, amit a generált assembly összevetésével lehetne igazolni. A gyakorlati következmény mindenesetre az, hogy `-Os/-Oz` mellett a LEA titkosítása gyorsabb a SPECK-nél (57,7, illetve 62,3 CPB), `-O3` mellett pedig fordítva.

### 5.6.4. A mérések ismételhetősége

A SPECK és a LEA `-O3` értékei ciklusra megegyeztek a mérőprogram korábbi, más szerkezetű változatával végzett mérésekkel, az `-Os` és az `-Oz` pedig azonos kód mellett azonos eredményt adott. Ezek, valamint a körök közötti 0,022 % alatti eltérés azt mutatják, hogy a mérési módszer megbízható, és a fenti különbségek a kódból, nem a mérésből származnak. Ez a kontrollált benchmarkra érvényes, nem valós üzemi terhelésre.

## 5.7. Az eredmények összegzése

`-O3` mellett 1024 bájtos üzeneten az ARX-alapú SPECK és LEA fajlagos költsége titkosításkor 3,7–4,5-ször, visszafejtéskor 8,7–10-szer kisebb, mint a vizsgált `tiny-AES-c` implementációé ECB módban. Az ASCON-128 a beépített hitelesítés ellenére kb. 80 bájt fölött gyorsabb a hitelesítést nem végző ECB és CTR módú AES-nél, rövid üzeneteknél viszont a két 12 körös permutáció kb. 6 400 ciklusos fix költsége a hátrányára szól. A legjobb fordítási szint algoritmusfüggő: az AES-nek az `-O3`, az ASCON-nak és a LEA titkosításának az `-Os/-Oz` kedvez, a SPECK-et alig befolyásolja. Egy adott alkalmazásban ezért nemcsak az algoritmust, hanem a fordítási beállítást is mérés alapján érdemes megválasztani.

# 6. fejezet

# Biztonsági elemzés és összehasonlító értékelés

A teljesítmény mellett legalább ilyen fontos, hogy egy algoritmus milyen védelmet nyújt az ismert támadásokkal szemben. Ez a fejezet a szakirodalom alapján foglalja össze a négy algoritmus biztonsági jellemzőit; támadásokat nem valósítottunk meg.

## 6.1. Alapfogalmak

### 6.1.1. Támadási modellek

A támadó képességei szerint négy, egyre erősebb modellt szokás megkülönböztetni [1]. A *csak titkosított szöveget ismerő* (ciphertext-only) támadó kizárólag a titkosított kimenetet látja. Az *ismert nyílt szöveges* (known-plaintext) modellben nyílt és titkosított szövegpárok is rendelkezésére állnak. A *választott nyílt szöveges* (chosen-plaintext, CPA) modellben maga választhatja meg a titkosítandó üzeneteket; ez reálisabb, mint elsőre tűnik, mert sok protokollban a támadó rá tudja venni az eszközt bizonyos üzenetek titkosítására. A *választott titkosított szöveges* (chosen-ciphertext, CCA) modellben a visszafejtendő szövegeket is ő választja. A modern algoritmusoktól általában a CPA- vagy a CCA-modellben várnak el biztonságot.

### 6.1.2. Biztonsági szint és biztonsági tartalék

Egy algoritmus *biztonsági szintje* $s$ bit, ha a legjobb ismert támadás $2^s$ műveletet igényel. Ideális esetben ez a kulcs hossza, vagyis a legjobb támadás a kulcstér végigpróbálása, ami 128 bites kulcsnál $2^{128} \approx 3{,}4 \times 10^{38}$ művelet.

A *biztonsági tartalék* (security margin) azt fejezi ki, hogy az algoritmus teljes körszáma ($N$) mennyivel több a legjobb ismert támadás által elért körszámnál ($n$): $N - n$ kör, vagy

arányosan $(N-n)/N$. Minél nagyobb a tartalék, annál több tér marad a jövőbeli, erősebb támadások számára [7]. Eltérő körszámú algoritmusokat csak az arányos értékkel lehet összevetni.

## 6.2. A főbb támadástípusok

### 6.2.1. Differenciális kriptoanalízis

A differenciális kriptoanalízist Biham és Shamir vezette be 1991-ben a DES elemzésére [26]. A támadó olyan nyíltszöveg-párokat választ, amelyek között ismert (általában XOR-) különbség van, és azt vizsgálja, hogyan terjed ez a különbség a körökön át. Ha bizonyos bemeneti különbségek a véletlennél nagyobb valószínűséggel adnak bizonyos kimeneti különbségeket, abból a körkulcsok egyes bitjeire lehet következtetni. A tervezők ezért úgy választják meg az S-boxokat és a körszámot, hogy a teljes titkosításon átívelő differenciális jellemzők valószínűsége elhanyagolható legyen. ARX-algoritmusoknál az elemzés nehezebb, mert a moduláris összeadás differenciális viselkedése az átvitelek miatt bonyolultabb, mint egy S-boxé.

### 6.2.2. Lineáris kriptoanalízis

Matsui 1993-ban publikálta a lineáris kriptoanalízist, szintén a DES-re [27]. A támadó a nyílt szöveg, a titkosított szöveg és a kulcs bitjei közötti olyan lineáris (XOR-) összefüggéseket keres, amelyek a véletlennél gyakrabban teljesülnek; elég sok ismert szövegpárból ezekkel kulcsbitek nyerhetők ki. Bogdanov és Rijmen később bevezették a nulla korrelációjú lineáris kriptoanalízist [20], a differenciális és a lineáris módszer kombinációja, a differenciális-lineáris támadás pedig az ARX-algoritmusok ellen az egyik leghatékonyabbnak bizonyult [22].

### 6.2.3. Algebrai és rotációs támadások

Az algebrai támadások az algoritmust $GF(2)$ feletti egyenletrendszerként írják fel, és ennek megoldásával próbálják meghatározni a kulcsot. A teljes körszámú algoritmusok egyenletrendszere azonban túl nagy a mai megoldók (Gröbner-bázisok, SAT-megoldók) számára, az ARX-algoritmusoknál pedig a moduláris összeadás bitszintű leírása még bonyolultabb egyenletekhez vezet [7].

A Khovratovich és Nikolić által bevezetett rotációs kriptoanalízis [28] kifejezetten ARX-szerkezetek ellen készült. Azt használja ki, hogy elforgatott bemenetre egy ideális ARX-rendszer kimenete is elforgatva jelenne meg; ezt az összefüggést a moduláris össze-

adás és a körkonstansok megtörik, a támadás pedig azt vizsgálja, mennyire. A SPECK és a LEA teljes változata ellen ez a módszer sem vezetett eredményre.

### 6.2.4. Oldalcsatorna-támadások

Az oldalcsatorna-támadások nem az algoritmus matematikai szerkezetét, hanem a fizikai megvalósítást használják ki [29]: a végrehajtási idő, az áramfelvétel (egyszerű és differenciális teljesítményanalízis, SPA és DPA), az elektromágneses kisugárzás vagy a gyorsítótár-hozzáférések mintázata információt hordozhat a kulcsról.

Az *időzítéses* és *gyorsítótár-alapú* támadások ellen a konstans idejű kód véd. A táblás AES-megvalósításokban az S-box-olvasás címe a kulcstól függ, így gyorsítótárral rendelkező rendszereken a hozzáférési idő elvben árulkodó lehet. Az ARX-algoritmusok és az ASCON bitsliced S-boxa nem használ adatfüggő memóriacímet vagy elágazást, ezért természetüknél fogva konstans idejű. A *teljesítményanalízis* ellen viszont a konstans idejűség nem véd: ehhez maszkolás (a köztes értékek véletlen részekre bontása) szükséges. A maszkolás az alacsony fokú S-boxot használó ASCON-nál viszonylag olcsó, ez a tervezők kifejezett célja volt [9]; az ARX-algoritmusoknál a moduláris összeadás, az AES-nél a nagy S-box miatt drágább.

Oldalcsatorna-méréseket nem végeztünk, mert ehhez oszcilloszkóp és speciális mérőeszközök szükségesek; ez a szakasz a szakirodalmi áttekintés része.

### 6.2.5. Születésnapi korlát

A születésnapi paradoxon miatt $n$ bites blokkméret mellett nagyjából $2^{n/2}$ blokk feldolgozása után nagy valószínűséggel ismétlődik egy titkosított blokk. CBC módban például két titkosított blokk egyezéséből a megfelelő nyíltszöveg-blokkok XOR-ja kiszámítható, és a CBC-MAC is hamisíthatóvá válik. A vizsgált blokktitkosítók 128 bites blokkmérete mellett ez a korlát $2^{64}$ blokk, ami IoT-alkalmazásokban gyakorlatilag elérhetetlen. Ezért vizsgáltuk a 64 bites blokkméretű változatok helyett a 128 biteseket.

## 6.3. Az algoritmusok biztonsági jellemzői

### 6.3.1. AES-128

Az AES-128 a legalaposabban vizsgált blokktitkosítók egyike. A legjobb ismert kulcsvisszafejtő támadások (lehetetlen differenciális és meet-in-the-middle támadások) a 10 körből 7-et érnek el [38], ami 3 kör, azaz 30 % tartalék. A teljes algoritmus ellen a legjobb ismert eljárás a biclique-támadás, amely a kulcstér végigpróbálásához képest mindössze

kb. négyszeres gyorsulást ér el ($2^{126,1}$ művelet), így gyakorlati jelentősége nincs [19].

### 6.3.2. SPECK 128/128

A SPECK 128/128 ellen a legjobb publikált támadás differenciális, és a 32 körből 23-at ér el, közel a kulcstér végigpróbálásának költségével [21]. Ez 9 kör, azaz kb. 28 % tartalék; a tervezők a körszámot kifejezetten úgy választották meg, hogy a tartalék az AES-éhez hasonló legyen [13]. Rotációs támadás a teljes változat ellen nem ismert.

### 6.3.3. LEA-128

A LEA-128 ellen a legjobb ismert támadás differenciális-lineáris, és a 24 körből 17-et ér el [22]. Ez 7 kör, azaz kb. 29 % tartalék. A tervezők részletes biztonsági elemzést közöltek a forgatási paraméterek megválasztásáról és a differenciális, illetve lineáris tulajdonságokról [15], az algoritmus pedig ISO/IEC-szabvány része [16].

### 6.3.4. ASCON-128

Az ASCON biztonságát AEAD-sémaként kell értékelni. A tervezők a titkosságra és a hitelességre egyaránt 128 bites biztonságot állítanak. A leghosszabb ismert kulcsvisszafejtő támadások a 12 körös inicializálásból 7 kört érnek el, ami 5 kör, azaz kb. 42 % tartalék [9, 40]. A nonce-ismétlést nem igénylő legjobb ilyen támadás kb. $2^{123}$ műveletet és $2^{64}$ adatot igényel [40]. Ez az eredmény az ASCON-128-ra és a 128a-ra egyaránt vonatkozik, mert inicializálásuk azonos. Az ASCON-128 256 bites capacity-je ráadásul nagyobb tartalékot hagy, mint a 128a 192 bites capacity-je. Megvalósítási szempontból előnyös, hogy konstans idejű, és viszonylag olcsón maszkolható.

Mint minden nonce-alapú AEAD-sémánál, az ASCON-nál is kritikus, hogy ugyanazzal a kulccsal egy nonce ne ismétlődjön: ismétlődés esetén a két üzenet nyílt szövegeinek XOR-ja kiszámítható, és a hitelesség sem garantált. Ez a séma használati feltétele, amelyet az alkalmazásban (például számlálóalapú nonce-szal) kell biztosítani; a mérésekben használt csupa nulla nonce csak mérési célt szolgált.

## 6.4. Összehasonlító értékelés

A 6.1. táblázat a fenti jellemzőket foglalja össze.

A három blokktitkosító arányos biztonsági tartaléka hasonló (28–30 %), az ASCON inicializálásáé ennél nagyobb. Ebből csak annyi következik, hogy a publikált támadások egyik blokktitkosító teljes körszámához sem jutottak lényegesen közelebb, mint a többiéhez. Azonos biztonsági szintet ez nem bizonyít. A támadások modellje, adat- és időigénye

6.1. táblázat. A vizsgált algoritmusok biztonsági jellemzői a szakirodalom alapján

| **Szempont** | **AES-128** | **SPECK 128/128** | **LEA-128** | **ASCON-128** |
|---|---|---|---|---|
| Állított biztonság | 128 bit | 128 bit | 128 bit | 128 bit |
| Legjobb támadás (kör) | 7 / 10 | 23 / 32 | 17 / 24 | 7 / 12[a] |
| Támadás típusa | lehet. diff., MITM | differenciális | diff.-lineáris | cube-típusú[a] |
| Biztonsági tartalék | 30 % | 28 % | 29 % | 42 % |
| Konstans idejű kód | nehéz[b] | természetes | természetes | természetes |
| Maszkolás költsége | nagy | nagy | nagy | kisebb |
| Szabvány | FIPS 197 | ISO/IEC 29167-22[d] | ISO/IEC 29192-2 | SP 800-232[c] |

[a]Az inicializáló permutáció 12 köréből.

[b]Táblás megvalósításban; bitsliced szoftveres vagy hardveres megvalósítással megoldható.

[c]A szabványos Ascon-AEAD128 az ASCON-128a változatból készült.

[d]Csak RFID-légi interfészhez.

A támadások modellje, adat- és időigénye eltér, ezért a tartalékok csak tájékoztató jellegűek.

eltér: a SPECK 23 körös támadása például kb. $2^{125}$ választott nyílt szöveget igényel [21], az AES 7 körös támadásai viszont $2^{100}$ alatti adattal és idővel dolgoznak [38]. Az AES-t ráadásul jóval régebben és többen elemzik, mint a két ARX-algoritmust. A gyakorlati különbségek inkább a megvalósításban jelentkeznek. Az ARX-algoritmusok és az ASCON konstans idejű kódja egyszerűbb, mint a táblás AES-é, a teljesítményanalízis elleni maszkolás pedig az ASCON-nál a legolcsóbb. A SPECK hátránya nem kriptoanalitikai, hanem elfogadottsági: az általános célú könnyűsúlyú blokktitkosító-szabványba nem került be, és tervezésének körülményei vitatottak.

# 7. fejezet

# Összefoglalás és továbbfejlesztési lehetőségek

## 7.1. A célkitűzések teljesülése

A dolgozat célja négy, eltérő felépítésű kriptográfiai algoritmus – az AES-128, a SPECK 128/128, a LEA-128 és az ASCON-128 – futásidejű költségének összevetése volt egy valós, ARM Cortex-M4 magos mikrovezérlőn (STM32L432KC). Az algoritmusokat egységes szerkezetű, hivatalos tesztvektorokkal ellenőrzött C-implementációkban valósítottuk meg, és közös, DWT-alapú, ciklusra pontos mérőprogramot írtunk hozzájuk. A kulcsgenerálás, a titkosítás és a visszafejtés költségét négy üzenetméreten és hat optimalizációs szinten, szintenként tíz ismétléssel mértük. Meghatároztuk a kódméretet és a veremigényt is, az eredményeket pedig a szakirodalmi biztonsági elemzéssel együtt értékeltük.

## 7.2. Fő megállapítások

**Az ARX-alapú algoritmusok fajlagos költsége lényegesen kisebb.** `-O3` mellett a SPECK 56–62, a LEA 49–76 ciklust igényelt bájtonként, a vizsgált AES-implementáció 276–492-t; ez titkosításkor 3,7–4,5-szörös, visszafejtéskor 8,7–10-szeres különbség. A SPECK kódja ráadásul optimalizált szinteken 300 bájt alatti.

**Az AES ECB-visszafejtése jelentős többletköltséggel jár.** 1024 bájton optimalizálás nélkül a visszafejtés 9,1-szer, `-O3` mellett 1,75-ször drágább a titkosításnál. Az ok az implementáció inverz MixColumns lépése; CTR módban a két irány költsége azonos, így a többlet megfelelő üzemmóddal elkerülhető.

**Az ASCON-128 hitelesítéssel együtt is versenyképes.** Költsége jó közelítéssel egy kb. 6 400 ciklusos fix tag (a két 12 körös permutáció) és bájtonként kb. 200 ciklus

összege. Ennek megfelelően `-O3` mellett kb. 80 bájt, `-Os/-Oz` mellett már kb. 23 bájt fölött gyorsabb a hitelesítést nem végző AES-nél, és nincs szüksége külön kulcsgenerálásra.

**A fordítási beállítás hatása nagy és algoritmusfüggő.** Az optimalizálás az AES-visszafejtést 34,8-szeresére gyorsította, az `-Ofast` sehol nem hozott 0,5 %-nál nagyobb változást. Az `-Os` és az `-Oz` azonos kódot adott. Méretre optimalizálva az ASCON 6 %-kal, a LEA titkosítása 24 %-kal gyorsabb, az AES 58 %-kal lassabb lett, a SPECK-re alig hatott. Az ASCON-nál ezt a kibontott permutáció mérete magyarázza, amely nem fér el az ART-gyorsító 1 KB-os utasítás-gyorsítótárában.

**A publikált támadások nem jeleznek gyengeséget.** A szakirodalom alapján a három blokktitkosító arányos biztonsági tartaléka hasonló (28–30 %), és egyik ellen sem ismert gyakorlatban kivitelezhető támadás. Ez nem jelent bizonyítottan azonos biztonsági szintet, mert a támadások feltételei eltérnek, de nem utal arra, hogy a könnyűsúlyú blokktitkosítók teljesítményelőnye a biztonság rovására menne.

## 7.3. Alkalmazási ajánlások

A 7.1. táblázat a mérések és a biztonsági elemzés alapján tipikus helyzetekre fogalmaz meg ajánlásokat. Az ajánlások 32 bites Cortex-M4 platformon mért adatokon alapulnak, más architektúrákra csak óvatosan vihetők át.

7.1. táblázat. Alkalmazási ajánlások 32 bites mikrovezérlőkre

| **Helyzet** | **Ajánlás** | **Indoklás** |
|---|---|---|
| Titkosság és hitelesség is szükséges | ASCON (új fejlesztésnél a szabványos Ascon-AEAD128) | Egy sémában adja a titkosítást és a hitelesítést; a mért ASCON-128 kb. 80 bájt fölött (`-Os` mellett már kb. 23 bájt fölött) a hitelesítés nélküli szoftveres AES-nél is gyorsabb. Érdemes méretre optimalizálva fordítani. A szabványos változatot nem mértük. |
| Nagy feldolgozási sebesség, a hitelesítést más réteg biztosítja | LEA-128 vagy SPECK 128/128 | A legkisebb fajlagos költség (1 000–1 650 kB/s); a LEA szabványos, a SPECK kódja a legkisebb, és kevésbé érzékeny a fordítási beállításra. |
| Nagyon rövid üzenetek, gyakori kulcscsere | SPECK 128/128 vagy LEA-128 | Nincs üzenetenkénti fix költség, és a kulcsgenerálás is olcsó (kb. 800–1100 ciklus). |
| Szabványmegfelelés (pl. FIPS) kötelező vagy van hardveres AES-gyorsító | AES-128 | Hardveres gyorsítóval a szoftveres hátrány megszűnik; szoftveresen CTR-re épülő módban (pl. GCM, CCM) érdemes használni, mert az inverz blokkművelet drága. |

A SPECK és a LEA önmagában csak titkosságot nyújt; ha hitelesség is kell, külön MAC-ra vagy hitelesített üzemmódra van szükség, ami a költségüket növeli.

## 7.4. Korlátok

- A mérések egyetlen platformon (STM32L432KC, Cortex-M4) készültek; más architektúrákon, különösen 8 és 16 bites eszközökön, az arányok eltérhetnek.
- Az AES-eredmények a méretre optimalizált `tiny-AES-c` implementációra érvényesek, az ASCON-nál pedig a referenciakódon alapuló megvalósítást mértük; optimalizált (például bitsliced vagy bit-interleaved, assembly) változatokkal a különbségek kisebbek lehetnek.
- A blokktitkosítókat ECB módban, az ASCON-t AEAD-ként mértük, így az utóbbi költsége a hitelesítést is tartalmazza; hitelesített AES-módot (GCM, CCM) nem mértünk.
- Csak blokkhatárra eső üzenetméreteket (16, 64, 256, 1024 bájt) mértünk, a részleges blokkok kezelésének költségét nem.
- Az ASCON-128-at mértük, nem a NIST SP 800-232 szerinti Ascon-AEAD128-at; erre csak becslést adtunk.
- A kódméretet a mérésnél használttól eltérő fordítóverzióval határoztuk meg.
- Az ART-gyorsítóra vonatkozó magyarázatot kikapcsolt gyorsítótárral végzett méréssel nem ellenőriztük.
- Energiafogyasztást és oldalcsatorna-szivárgást nem mértünk; a biztonsági értékelés szakirodalmi adatokon alapul.

## 7.5. Továbbfejlesztési lehetőségek

**A szabványos Ascon-AEAD128 mérése.** A NIST SP 800-232 szerinti változat megvalósítása és mérése megmutatná, mennyire pontos a 5.3. alfejezetben adott becslés, és a szabványos sémát a többi algoritmussal közvetlenül is összevethetővé tenné.

**A gyorsítótár hatásának igazolása.** Az ART utasítás-gyorsítótárának kikapcsolásával, illetve az előolvasás bekapcsolásával végzett mérések eldönthetnék, hogy az ASCON `-Os/-Oz` melletti előnyét valóban a kódméret okozza-e. A LEA irányfüggő viselkedésének okát a generált assembly összevetése tárhatja fel.

**Optimalizált implementációk.** Cortex-M-re optimalizált AES- és ASCON-megvalósítások mérése megmutatná, mennyi a különbség az algoritmusok és mennyi a megvalósítási stílus között.

**További platformok.** A mérések kiterjesztése más Cortex-M magokra (M0+, M33), valamint RISC-V és AVR mikrovezérlőkre megmutatná, mennyire általánosíthatók az eredmények.

**Energia- és oldalcsatorna-mérés.** Precíziós árammérővel az algoritmusok tényleges energiaigénye, oszcilloszkóppal pedig a teljesítményanalízissel szembeni ellenállásuk is vizsgálható lenne.

**Azonos funkcionalitású összevetés.** Az AES és a LEA hitelesített üzemmódjainak (például GCM, CCM) mérése lehetővé tenné, hogy az ASCON-t azonos szolgáltatást nyújtó megoldásokkal hasonlítsuk össze.

# Irodalomjegyzék


[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.

[2] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer, 2010.

[3] C. E. Shannon, *Communication Theory of Secrecy Systems*, Bell System Technical Journal **28** (1949), no. 4, 656–715.

[4] NIST, *Advanced Encryption Standard (AES), FIPS PUB 197*, National Institute of Standards and Technology, 2001.

[5] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer, 2002.

[6] K. A. McKay, L. Bassham, M. S. Turan, and N. Mouha, *Report on Lightweight Cryptography*, NIST Internal Report 8114 (2017).

[7] A. Biryukov and L. Perrin, *State of the Art in Lightweight Symmetric Cryptography*, IACR ePrint Archive 2017/511 (2017).

[8] G. Hatzivasilis, K. Fysarakis, I. Papaefstathiou, and C. Manifavas, *A Review of Lightweight Block Ciphers*, Journal of Cryptographic Engineering **8** (2018), no. 2, 141–184.

[9] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology **34** (2021), no. 3.

[10] NIST, *Lightweight Cryptography Standardization Process: NIST Selects Ascon*, National Institute of Standards and Technology, 2023.

[11] NIST, *Ascon-Based Lightweight Cryptography Standards for Constrained Devices (NIST SP 800-232)*, National Institute of Standards and Technology, 2025. Available: https://csrc.nist.gov/pubs/sp/800/232/final.

[12] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *The SIMON and SPECK Families of Lightweight Block Ciphers*, IACR ePrint Archive 2013/404 (2013).

[13] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *Notes on the Design and Analysis of SIMON and SPECK*, IACR ePrint Archive 2017/560 (2017).

[14] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *SIMON and SPECK Implementation Guide, Version 1.1*, National Security Agency, 2019.

[15] D. Hong, J.-K. Lee, D.-C. Kim, D. Kwon, K. H. Ryu, and D.-G. Lee, *LEA: A 128-Bit Block Cipher for Fast Encryption on Common Processors*, WISA 2013, Lecture Notes in Computer Science **8267** (2013), 3–27.

[16] ISO/IEC, *Information security – Lightweight cryptography – Part 2: Block ciphers (ISO/IEC 29192-2:2019)*, International Organization for Standardization, 2019.

[17] D. Dinu, A. Biryukov, J. Großschädl, D. Khovratovich, Y. Le Corre, and L. Perrin, *Triathlon of Lightweight Block Ciphers for the Internet of Things*, NIST Lightweight Cryptography Workshop (2015).

[18] M. Cazorla, K. Marquet, and M. Minier, *Survey and Benchmark of Lightweight Block Ciphers for Wireless Sensor Networks*, SECRYPT 2013 (2013), 543–548.

[19] A. Bogdanov, D. Khovratovich, and C. Rechberger, *Biclique Cryptanalysis of the Full AES*, ASIACRYPT 2011, Lecture Notes in Computer Science **7073** (2011), 344–371.

[20] A. Bogdanov and V. Rijmen, *Linear Hulls with Correlation Zero and Linear Cryptanalysis of Block Ciphers*, Designs, Codes and Cryptography **70** (2014), no. 3, 369–383.

[21] L. Song, Z. Huang, and Q. Yang, *Automatic Differential Analysis of ARX Block Ciphers with Application to SPECK and LEA*, ACISP 2016, Lecture Notes in Computer Science **9723** (2016), 379–394.

[22] Y. Chen, Z. Bao, and H. Yu, *Differential-Linear Approximation Semi-Unconstrained Searching and Partition Tree: Application to LEA and Speck*, ASIACRYPT 2023 (IACR ePrint Archive 2023/1414) (2023).

[23] M. M. Hossain, M. Fotouhi, and R. Hasan, *Towards an Analysis of Security Issues, Challenges, and Open Problems in the Internet of Things*, IEEE World of Wireless, Mobile and Multimedia Networks (WoWMoM) (2015), 1–6.

[24] Ericsson, *Ericsson Mobility Report June 2026*, Telefonaktiebolaget LM Ericsson, 2026. Available: https://www.ericsson.com/mobility-report.

[25] A.-R. Sadeghi, C. Wachsmann, and M. Waidner, *Security and Privacy Challenges in Industrial Internet of Things*, 52nd ACM/EDAC/IEEE Design Automation Conference (DAC) (2015), 1–6.

[26] E. Biham and A. Shamir, *Differential Cryptanalysis of DES-like Cryptosystems*, Journal of Cryptology **4** (1991), no. 1, 3–72.

[27] M. Matsui, *Linear Cryptanalysis Method for DES Cipher*, EUROCRYPT '93, Lecture Notes in Computer Science **765** (1993), 386–397.

[28] D. Khovratovich and I. Nikolić, *Rotational Cryptanalysis of ARX*, FSE 2010, Lecture Notes in Computer Science **6147** (2010), 333–346.

[29] P. C. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, CRYPTO '96, Lecture Notes in Computer Science **1109** (1996), 104–113.

[30] D. Lee, D.-C. Kim, D. Kwon, and H. Kim, *Efficient Hardware Implementation of the Lightweight Block Encryption Algorithm LEA*, Sensors **14** (2014), no. 1, 975–994.

[31] KISA, *LEA (Lightweight Encryption Algorithm): algoritmusleírás és tesztvektorok*, Korea Internet & Security Agency. Available: https://seed.kisa.or.kr.

[32] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Methods and Techniques (NIST SP 800-38A)*, National Institute of Standards and Technology, 2001.

[33] kokke, *tiny-AES-c: Small portable AES128/192/256 in C*, GitHub. Available: https://github.com/kokke/tiny-AES-c.

[34] The Ascon Team, *ascon-c: Ascon reference and optimized implementations in C*, GitHub. Available: https://github.com/ascon/ascon-c.

[35] STMicroelectronics, *RM0394 Reference Manual: STM32L41xxx/42xxx/43xxx/44xxx/45xxx/46xxx Advanced Arm-based 32-bit MCUs*, STMicroelectronics. Available: https://www.st.com.

[36] ARM, *Cortex-M4 Technical Reference Manual (ARM DDI 0439B)*, ARM Limited, 2010.

[37] Free Software Foundation, *Using the GNU Compiler Collection (GCC): Options That Control Optimization*, Free Software Foundation. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html.

[38] P. Derbez, P.-A. Fouque, and J. Jean, *Improved Key Recovery Attacks on Reduced-Round AES in the Single-Key Setting*, EUROCRYPT 2013, Lecture Notes in Computer Science **7881** (2013), 371–387.

[39] T. Ashur and A. Luykx, *An Account of the ISO/IEC Standardization of the Simon and Speck Block Cipher Families*, Security of Ubiquitous Computing Systems, Springer (2021), 63–78.

[40] R. Rohit, K. Hu, S. Sarkar, and S. Sun, *Misuse-Free Key-Recovery and Distinguishing Attacks on 7-Round Ascon*, IACR Transactions on Symmetric Cryptology **2021** (2021), no. 1, 130–155.

[41] ISO/IEC, *Information technology – Automatic identification and data capture techniques – Part 22: Crypto suite SPECK security services for air interface communications (ISO/IEC 29167-22:2018)*, International Organization for Standardization, 2018.

# Köszönetnyilvánítás

Köszönöm témavezetőmnek, Dr. Békési Józsefnek, hogy a témaválasztástól a dolgozat lezárásáig segítette a munkámat, és mindig hasznos észrevételekkel látott el.

Köszönöm a családomnak és a barátaimnak, hogy a tanulmányaim alatt végig mellettem álltak.

# Nyilatkozat

Alulírott, Kárász Barna Kristóf, mérnökinformatikus BSc szakos hallgató, kijelentem, hogy a szakdolgozatban ismertetettek saját munkám eredményei, és minden felhasznált, nem saját munkából származó eredmény esetén hivatkozással jelöltem annak forrását.

Szeged, 2026. október 15.

*Kárász Barna Kristóf*

# Mellékletek

A dolgozat elektronikus melléklete a mérések megismétléséhez szükséges összes fájlt tartalmazza.

**1. Forráskód** (STM32CubeIDE-projekt)

- `main.c` – önellenőrzés, mérési ciklus, statisztika, kiírás;
- `aes.c/h`, `speck.c/h`, `lea.c/h`, `ascon128.c/h` – az algoritmusok;
- `build_opt.h` – a fordítási szint ellenőrzése.

**2. Mérési naplók**

- a hat optimalizációs szint (`-O0`, `-O2`, `-O3`, `-Ofast`, `-Os`, `-Oz`) teljes soros kimenete, körönként CSV-sorokkal és összesítő táblázattal;
- `osszesito_median.csv` – a mediánok és a származtatott értékek (CPB, feldolgozási sebesség, idő) szintenként.

**3. Kiértékelés**

- `graphics.py` – a naplók feldolgozása és a 5. fejezet ábráinak előállítása (Python, pandas, matplotlib);
- az ábrák PDF-változata.